NOSE-CONE DESIGN TOOL: USER GUIDE

Fabio Feig - 315169



Date: October 11, 2026

Doc ID: 2026_C_FD_NOSE-CONE-TOOL_USER-GUIDE

Python package `nosecone` – version 1.0

EPFL ROCKET TEAM

Contents

Quick start 1

1 What the Tool Does, and What It Does Not 1
1.1 Purpose . . . 1
1.2 What is computed . . . 1
1.3 Models and their validity . . . 2
1.4 What the tool does not do . . . 3
1.5 Reproducibility . . . 3

2 Installation and Requirements 4
2.1 Requirements . . . 4
2.2 Installation . . . 4
2.3 Checking the installation . . . 4

3 Repository Structure 4
3.1 Folders . . . 5
3.2 Modules . . . 5
3.3 Using the package from Python . . . 6

4 The Configuration File 7
4.1 How the file works . . . 7
4.2 All parameters . . . 7
`project`: Project . . . 7
`vehicle`: Vehicle . . . 8
`engine`: Engine . . . 8
`fins`: Fins . . . 8
`launch`: Launch . . . 9
`atmosphere`: Atmosphere . . . 9
`flight`: Flight model . . . 9
`nose`: Nose: design space and selected design . . . 10
`aerodynamics`: Aerodynamics (wave-drag model) . . . 10
`payload`: Payload . . . 11
`structure`: Structure (mass model) . . . 11
`heating`: Heating . . . 11
`bluntness`: Tip bluntness . . . 12
`robustness`: Robustness . . . 12
`trade_matrix`: Decision matrix . . . 13
`validation`: CFD validation plots . . . 13
`cfd`: CFD solver and studies . . . 13
`cad`: CAD export . . . 14
`output`: Outputs . . . 14
4.3 Structured parameters . . . 15

5 Running Studies **17**
5.1 Commands . . . 17
5.2 Worked example 1: a single design . . . 17
5.3 Worked example 2: a parameter sweep . . . 19
5.4 The full Weisshorn study . . . 20
5.5 Another rocket . . . 21
5.6 New CFD cases . . . 21

6 Reading the Outputs **21**
6.1 The output folder . . . 21
6.2 Tables . . . 22
6.3 Plots . . . 23

7 Extending the Tool **32**
7.1 Adding a profile family . . . 32
7.2 Adding a plot . . . 33
7.3 Adding a configuration parameter . . . 34
7.4 Changing a physical model . . . 34
7.5 Checking a change . . . 34

A Verification Record **35**

B Troubleshooting **35**

Quick start

From zero to the Weisshorn results in four commands (Windows; see Section 2 for macOS/Linux):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m nosecone run configs/weisshorn.yaml
```

Then open `output/weisshorn/summary.md`: it lists every nose of the study, best first, with its wave drag, apogee, mass, payload room and static-margin change. Figures are in `output/weisshorn/figures/`, CAD files (CSV, STL, STEP) in `output/weisshorn/cad/`.

To study another rocket, copy `configs/example_new_rocket.yaml`, change the values of your vehicle (diameter, length, masses, engine, fins) and the noses you want to compare, and run it with the same command. You never need to edit the Python code for that: every input is in the YAML file. The parameters are described in Section 4, the outputs in Section 6.

How this guide is organised.

- Section 1: what the tool computes, with which models, and where these models stop being valid.
- Section 2: installation and requirements.
- Section 3: the repository and what each module does.
- Section 4: every parameter of the configuration file (meaning, unit, valid range, default).
- Section 5: running a single design and a parameter sweep, with worked examples for our 180 mm rocket.
- Section 6: how to read every plot and table.
- Section 7: adding a profile family, a plot, a parameter, or changing a model.

1 What the Tool Does, and What It Does Not

1.1 Purpose

The `nosecone` package helps to choose the outer shape of the nose cone of a supersonic sounding rocket: its *profile* (cone, ogive, Von Kármán, ...), its *fineness ratio* L/D (length over diameter) and its *tip radius*. It compares candidates on the quantities a design review asks for (apogee, mass, payload room, stability, heating, manufacturability) and produces the CAD geometry of the selected design. It was written for the Weisshorn nose cone (2026_C_FD_REPORT-1_NOSE_CONE) and reproduces that report exactly, but every vehicle-specific number is an input, so the same code serves any ERT rocket.

1.2 What is computed

Figure 1 shows the chain. Everything starts from a YAML configuration file and a database of nose wave-drag coefficients computed with the tool's own CFD solver.

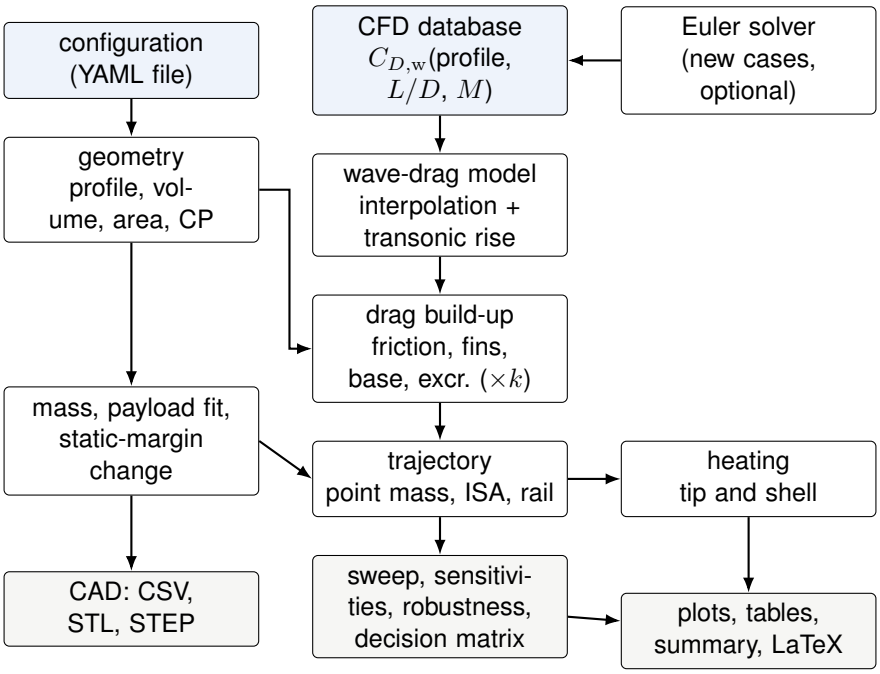


Figure 1: Computation chain of one run (`python -m nosecone run`). Shaded blue: inputs; grey: outputs.

For every nose of the sweep (each profile $\times$ each fineness $\times$ each tip radius) the tool computes:

- **Geometry**: profile $y(x)$, internal volume, wetted area, angle at the shoulder, tip angle, Barrowman centre of pressure, length over which the payload fits.
- **Wave drag** $C_{D,w}(M)$: the pressure drag of the nose caused by its shock and expansion waves, read from the CFD database and interpolated.
- **Apogee**: a full ascent with the drag of the whole vehicle, the nose length and the nose mass included.
- **Mass** of the shell and tip, **payload fit** and nose-bay length, **static-margin change** relative to a baseline nose.

For the study as a whole: apogee **sensitivities** (per kg, per 0.01 of C_D), the **robustness** of the ranking to the uncertain assumptions, the **cost of a blunt tip**, the **temperatures** of the tip and the shell on a nominal and a hot-day flight, a weighted **decision matrix**, and the **CAD files** of the selected design.

1.3 Models and their validity

Table 1 lists the models. They are deliberately simple and fast (a full study takes minutes), which is right for comparing shapes, but each has a range outside which its numbers should not be trusted.

Table 1: Models of the tool and their range of validity.

Quantity	Model	Valid for / limitations
Wave drag	Axisymmetric Euler CFD (inviscid, zero angle of attack), tabulated at $M = 1.1$–1.6 and $L/D = 3$–5 (Weisshorn database)	Supersonic, $1.1 \leq M \lesssim 2$; slender noses. Interpolated in M and in L/D (on $C_{D,\mathrm{w}}(L/D)^2$); *held constant* outside the computed range. No viscous effects.
Transonic drag	Smooth (cubic) rise from 0 at $M = 0.8$ to the $M = 1.1$ value at $M = 1$	An assumption, the weakest part of the chain; checked by the robustness study. Replace by RANS data if available.
Rest of the vehicle	Empirical build-up: turbulent flat-plate friction (roughness-limited, compressibility corrected), fin pressure drag (Ackeret), OpenRocket base drag, ERT-Sim excrescences; one factor k	Order-of-magnitude accuracy only, hence the calibration of k on a known apogee. Trapezoidal fins, no boat tail, no protuberances.
Trajectory	2D point mass, rail, constant thrust, linear mass decrease, ISA troposphere, no wind	Apogee estimates and comparisons. No rotation, no 6-DOF, no wind. The atmosphere is valid below ≈ 11 km; above that use a better model.
Stability	Barrowman (subsonic, linear), nose $C_{N\alpha} = 2$	*Changes* of static margin between noses; the absolute margin needs the full vehicle (fins, CG) in RocketPy/OpenRocket.
Heating	Fay–Riddell stagnation point; turbulent flat plate with Eckert reference temperature; 1D transient conduction, adiabatic inner face; lumped metal tip	Conservative peak temperatures for $M \lesssim 3$. No radiation, no internal convection, no transition, no ablation.
Mass	Wetted area $\times$ areal density + tip mass	Scaled from an as-built nose; no coupler, payload or bulkhead.

1.4 What the tool does not do

- **No structural analysis**: no stresses, buckling, flutter or layup design; the mass is an estimate.
- **No viscous or transonic CFD**: the solver is inviscid and supersonic. For the final design, run RANS (e.g. Fluent with k-ω SST) at a few Mach numbers.
- **No angle of attack**: normal force and centre of pressure come from Barrowman's formulas, not CFD.
- **No full-vehicle flight simulation**: use RocketPy with the exported $C_{D,\mathrm{w}}(M)$ and geometry.
- **No automatic optimisation**: the tool evaluates the noses you list; it does not search for a new shape.
- **No new CFD unless asked**: a profile, fineness or tip radius missing from the database is skipped with a message, unless `aerodynamics.compute_missing` is true (then the Euler solver runs, several minutes per case).

1.5 Reproducibility

The tool was refactored from the scripts that produced the Weisshorn design report and checked against them: with `configs/weisshorn.yaml` it reproduces the report's tables, quoted numbers, CAD files and figures exactly (Appendix A). `tests/verify_reference.py` repeats that check; run it after any change to the code.

2 Installation and Requirements

2.1 Requirements

- **Python 3.11 or newer** (developed with 3.14) and `git` to get the repository.
- The Python packages of `requirements.txt`: NumPy, SciPy, pandas, Matplotlib, PyYAML and, optionally, gmsh. The first four are pinned to the versions that produced the Weisshorn report, so that the results are reproduced bit for bit; newer versions give the same results to about 10^{-12}.
- **Optional: a LaTeX installation** (MiKTeX on Windows, TeX Live on Linux/macOS). With it, the plots use LaTeX fonts and are identical to the report figures; without it, Matplotlib's built-in renderer is used automatically and the plots look slightly different but contain the same information.
- **Optional: gmsh** (installed by `requirements.txt`, ≈70 MB) for the STEP export. Without it the STEP file is skipped with a message; the CSV and STL files are always written.

2.2 Installation

Get the repository (or download it as a zip from GitHub) and install the packages in a virtual environment, so that they do not interfere with other Python projects:

```
git clone https://github.com/<owner>/weisshorn-nose-cone.git
cd weisshorn-nose-cone
python -m venv .venv
.venv\Scripts\activate              # Windows (PowerShell or cmd)
source .venv/bin/activate           # macOS / Linux
pip install -r requirements.txt
```

The virtual environment must be activated (the prompt shows `(.venv)`) in every new terminal before using the tool. All commands of this guide are run from the repository folder.

2.3 Checking the installation

```
python -m nosecone --version                    # prints: nosecone 1.0.0
python -m nosecone validate configs/weisshorn.yaml
python tests/verify_reference.py                # about 2 minutes; must end with "All
    outputs match"
```

Common problems. *"No module named nosecone"*: you are not in the repository folder, or the virtual environment is not active. *LaTeX errors in the plots*: set `output.text_renderer: mathtext` or install the missing LaTeX packages (MiKTeX installs them on the fly if allowed). *"STEP skipped"*: `pip install gmsh`.

3 Repository Structure

3.1 Folders

```
weisshorn-nose-cone/
|-- nosecone/              the Python package (all the code)
|   |-- plotting/          figures (style.py, figures.py)
|-- configs/               weisshorn.yaml + worked examples
|-- data/
|   |-- euler_results.csv  CFD database: wave drag of every case
|   |-- euler_sweep_log.csv   run log of the CFD solver
|   |-- cfd/cp/            wall pressure of every CFD point (.npz)
|   |-- cfd/fields/        flow fields at the design fineness
|-- plugins/               example plugin (Section 7.1)
|-- docs/user_guide/       this guide (LaTeX source and PDF)
|-- report/                the Weisshorn nose-cone design report
|-- tests/                 verify_reference.py + reference outputs
|-- legacy/scripts/        the original scripts (not used)
|-- output/                created by the tool (not in git)
|-- requirements.txt, README.md
```

3.2 Modules

Each module has one job (Table 2). The functions have docstrings and type hints: in Python, `help(nosecone.flight.simulate)` prints the documentation of any function.

Table 2: Modules of the `nosecone` package, in the order of the computation chain.

Module	Job	Main contents
`config`	Read, complete and check the YAML file	`load()`, the dataclasses of every section (parameters with unit, range, default, description), `describe()`
`constants`	Physical constants and model coefficients (the only fixed numbers)	gas constant, γ, Sutherland, friction, base-drag, heating coefficients, grid-generator constants
`geometry`	Profiles and their geometric properties	profile families and `register_family()`, `radius()`, blunting, `volume_and_area()`, CP, payload length
`euler`	Axisymmetric Euler CFD solver	`make_grid()`, `Mesh`, `EulerAxi` (HLLC, MUSCL, local time stepping)
`taylor_maccoll`	Exact cone flow (solver validation)	`cone_cp()`
`cfd`	CFD studies and the database	`run_jobs()` (parallel), `rebuild_database()`, `load_database()`
`aerodynamics`	Wave-drag model from the database	`WaveDrag` (interpolation in M and L/D, transonic rise)
`flight`	Atmosphere, drag build-up, trajectory	`Atmosphere`, `DragModel`, `simulate()`, `calibrate()`
`stability`	Barrowman stability, nose mass	`fin_cna_xcp()`, `static_margin_shift()`, `nose_mass()`
`heating`	Tip and shell temperatures	`h_stagnation()`, `h_turbulent()`, `wall_response()`, `tip_insert()`
`design`	The study: sweep and derived results	`Study`: `sweep()`, `sensitivities()`, `robustness()`, `bluntness()`, `heating()`, `trade_matrix()`, `catalogue()`
`export`	CAD and meshing files	profile CSV, `write_stl()`, `write_step()` (gmsh), `stl_check()`
`plotting`	All figures	`figures.PLOTS` (one function per plot), `PLOT_DEFAULTS`, `style`
`reporting`	Tables and summaries	`summary_markdown()`, LaTeX tables and macros
`pipeline`	Runs everything in order	`run()`
`cli`	Command line	`python -m nosecone <command>`
`plugins`, `docs`	Plugin loading; documentation helpers	`load_plugins()`; parameter tables of this guide

3.3 Using the package from Python

The command line covers normal use, but every step is also a function. For example, in a Jupyter notebook:

```
from nosecone import config, pipeline
cfg = config.load("configs/weisshorn.yaml")
cfg.nose.fineness = [4.0, 4.25, 4.5]          # change anything before running
res = pipeline.run(cfg, plots=["trade"])
res["trade"].sort_values("apogee")            # pandas DataFrame of the sweep
st = res["study"]                             # the Study object (wave drag, vehicles,
    ...)
st.wave_drag("vonkarman", 4.5)(1.3)           # C_D,w of a nose at M = 1.3
```

4 The Configuration File

4.1 How the file works

A study is described by one YAML file (`configs/weisshorn.yaml` is the complete, commented example). YAML is a plain-text format: *sections* contain *parameters* written `name: value`, indented by two spaces; lists are written `[a, b, c]`; everything after # is a comment.

```
vehicle:
  diameter: 0.180        # m
  mass_wet: 75.0         # kg
nose:
  profiles: [ogive, power050, vonkarman]
  fineness: [4.0, 4.5, 5.0]
```

- **Only what differs from the default is needed.** Every parameter has a default (Section 4.2); a file with only a `vehicle` section is valid.
- **Units are SI** (m, kg, s, N, K) unless the table says otherwise (°, °C). Write lengths in metres: `diameter: 0.180`, not 180.
- **Relative paths** (database, output folder, CFD folders, plugins) are relative to the file that contains them.
- **Inheritance**: a file that starts with `base: weisshorn.yaml` takes every value of that file and overrides only what it lists (both worked examples of Section 5 do this). Sections are merged parameter by parameter; a list given in the new file replaces the inherited list.
- **Checking**: `python -m nosecone validate my.yaml` reads the file and stops at the first problem with a precise message, e.g. `unknown parameter 'nose.profils' (did you mean 'profiles'?)` or `'launch.rail_elevation' = 95.0 is outside the valid range (0, 90]`. Add `--show` to print every value the run will use (defaults included). The same file is saved next to the results as `config_used.yaml`.
- **Plugins**: `project.plugins` lists Python files imported before the run; they can add profile families (Section 7.1).

4.2 All parameters

The tables below are generated from the code (`python -m nosecone describe-config --latex`), so they always match the installed version. "Valid" gives the accepted range or values; parameters marked * are advanced numerical settings that normally stay at their default. Parameters whose default is "see text" are structured values, described in Section 4.3.

`project`: Project

Parameter	Default	Unit	Valid	Meaning
`name`	`Weisshorn`	–	–	Vehicle or study name, used in plot titles and file names
`output_dir`	`output`	path	–	Folder for all outputs (created if missing)

Parameter	Default	Unit	Valid	Meaning
`plugins`	[]	path	–	Python files imported before the run, e.g. new profile families (see the user guide)

`vehicle`: Vehicle

Parameter	Default	Unit	Valid	Meaning
`diameter`	0.18	m	> 0	External (body) diameter D; the nose base diameter
`length`	5.4	m	> 0	Total vehicle length including the reference nose
`reference_nose_length`	0.72	m	> 0	Nose length included in ‘length‘; the body is length - reference_nose_length
`mass_wet`	75	kg	> 0	Lift-off mass
`mass_dry`	45	kg	> 0	Burn-out mass used by mass_model ’lvo’
`surface_roughness`	2e-05	m	>= 0	Equivalent sand roughness of the skin (painted CFRP ˜ 20 um)
`nozzle_exit_diameter`	0.075	m	>= 0	Nozzle exit diameter; removed from the base area while burning

`engine`: Engine

Parameter	Default	Unit	Valid	Meaning
`thrust`	4500	N	> 0	Thrust (constant over the burn)
`burn_time`	9.1	s	> 0	Burn duration
`total_impulse`	40960	N s	> 0	Total impulse (sets the propellant mass for mass_model ’isp’)
`specific_impulse`	200	s	> 0	Specific impulse (propellant mass = I_t / (I_sp g0))

`fins`: Fins

Parameter	Default	Unit	Valid	Meaning
`count`	4	–	[1, 8]	Number of fins
`root_chord`	0.3	m	> 0	Root chord
`tip_chord`	0.12	m	>= 0	Tip chord
`span`	0.16	m	> 0	Semi-span (root to tip)
`sweep`	0.18	m	>= 0	Leading-edge sweep length (root LE to tip LE, axial)
`thickness`	0.005	m	> 0	Fin thickness
`edge_fraction`	0.15	–	(0, 0.5]	Length of each LE/TE wedge of the hexagonal section, as a fraction of the mean chord

`launch`: Launch

Parameter	Default	Unit	Valid	Meaning
`rail_length`	12	m	> 0	Launch rail length
`rail_elevation`	84	°	(0, 90]	Rail elevation above the horizontal
`site_altitude`	0	m	[-500, 5000]	Launch site altitude above sea level

`atmosphere`: Atmosphere

Parameter	Default	Unit	Valid	Meaning
`model`	`isa_troposphere`	–	isa_troposphere	Atmosphere model
`ground_temperature`	288.15	K	[200, 340]	Temperature at the launch site (288.15 K = ISA)
`ground_pressure`	101325	Pa	> 0	Pressure at the reference altitude of the model
`lapse_rate`	0.0065	K/m	> 0	Temperature lapse rate in the troposphere
`tropopause_temperature`	216.65	K	> 0	Temperature floor (tropopause / lower stratosphere)

`flight`: Flight model

Parameter	Default	Unit	Valid	Meaning
`mass_model`	`isp`	–	isp, lvo	Nominal mass model: 'isp' (propellant = I_t/(I_sp g0)) or 'lvo' (wet -> dry)
`calibrate`	`true`	–	–	Scale the non-nose drag by a factor k so that the baseline reaches target_apogee
`target_apogee`	9000	m	> 0	Apogee the baseline nose must reach when calibrate is true
`drag_factor`	1	–	> 0	Factor k on the non-nose drag when calibrate is false
`calibration_bracket`*	`[0.3, 1.5]`	–	–	Search interval for k
`calibration_tolerance`*	0.0001	–	> 0	Absolute tolerance on k
`t_max`*	120	s	> 0	Integration time limit (apogee normally ends the integration earlier)
`max_step`*	0.02	s	> 0	Maximum ODE time step
`rtol`*	1e-08	–	> 0	ODE relative tolerance
`atol`*	1e-08	–	> 0	ODE absolute tolerance
`output_step`*	0.05	s	> 0	Output sampling of the trade-study trajectories (sets M_max)
`detail_output_step`*	0.02	s	> 0	Output sampling of the trajectories used for heating and plots
`envelope_mass_model`	`lvo`	–	isp, lvo	Mass model of the hot-day envelope trajectory

Parameter	Default	Unit	Valid	Meaning
envelope_ground_temperature	303.15	K	[200, 340]	Ground temperature of the hot-day envelope
mass_sensitivity_step *	1	kg	> 0	Mass step for the apogee sensitivity dz/dm
cd_sensitivity_step *	0.01	–	> 0	C_D step for the apogee sensitivity dz/dC_D

nose: Nose: design space and selected design

Parameter	Default	Unit	Valid	Meaning
profiles	see text	–	–	Profiles compared in the sweep (built-in names or keys of custom_profiles)
fineness	[3, 3.5, 4, 4.5, 5]	–	–	Fineness ratios L/D compared in the sweep
baseline_profile	vonkarman	–	–	Reference nose for apogee and static-margin changes
baseline_fineness	4	–	> 0	Fineness of the reference nose
baseline_label	baseline	–	–	How the reference length is called in table captions (e.g. 'LVS length')
design_profile	vonkarman	–	–	Selected profile (CAD export, heating, final drawing)
design_fineness	4.5	–	> 0	Selected fineness L/D
design_tip_radius_ratio	0.05	–	[0, 0.5]	Selected spherical tip radius r_n / R (0 = sharp)
tip_radius_ratios	[0]	–	–	Tip radii r_n / R compared in the sweep (needs CFD data for each)
shoulder_length	0.06	m	>= 0	Cylindrical shoulder behind the curved part (coupler bond)
custom_profiles	{}	–	–	Extra profiles: name -> {family: ..., <parameter>: value, label: ...}

aerodynamics: Aerodynamics (wave-drag model)

Parameter	Default	Unit	Valid	Meaning
database	data/euler_results.csv	path	–	CSV of Euler wave-drag results (non-dimensional, reusable for any diameter)
user_database	(empty)	path	–	Optional second CSV for cases computed with compute_missing (empty: they are added to 'database')
studies	[main, f45]	–	–	Database studies used for the wave-drag model
grid	medium	–	coarse, medium, fine	Grid level of the database rows used
min_fineness *	0	–	>= 0	Database rows below this fineness are ignored

Parameter	Default	Unit	Valid	Meaning
`max_fineness` *	5	–	> 0	Database rows above this fineness are ignored
`machs`	`[1.1, 1.2, 1.4, 1.6]`	–	–	Supersonic Mach numbers at which the wave drag is tabulated
`transonic_onset`	0.8	–	[0.5, 1]	Mach number where the nose wave drag starts to rise
`transonic_full`	1	–	[0.8, 1.2]	Mach number where the rise reaches the value at transonic_match
`transonic_match`	1.1	–	[1, 1.5]	Lowest Euler Mach number; the model is flat between transonic_full and this
`compute_missing`	`false`	–	–	Run the Euler solver for profiles / fineness / tip radii missing from the database
`missing_machs`	`[1.6, 1.4, 1.2, 1.1]`	–	–	Mach sequence for cases computed on demand (highest first)

`payload`: Payload

Parameter	Default	Unit	Valid	Meaning
`enabled`	`true`	–	–	Evaluate the payload fit and nose-bay length
`name`	3U	–	–	Payload name used in labels
`length`	0.3405	m	> 0	Payload length
`width`	0.1	m	> 0	Payload width (square section), used in drawings
`required_radius`	0.077	m	> 0	Outer nose radius needed to house the payload section (half-diagonal + clearance + guide + shell)
`axial_clearance`	0.01	m	>= 0	Axial margin for the payload bulkhead

`structure`: Structure (mass model)

Parameter	Default	Unit	Valid	Meaning
`shell_areal_density`	0.98	kg/m^2	> 0	Shell mass per wetted area (Firehorn as-built: 0.98)
`tip_mass`	0.06	kg	>= 0	Machined tip insert mass (same for all profiles)
`shell_cg_fraction` *	0.35	–	[0, 1]	Shell centre of gravity ahead of the shoulder, as a fraction of L

`heating`: Heating

Parameter	Default	Unit	Valid	Meaning
`enabled`	`true`	–	–	Run the heating analysis

Parameter	Default	Unit	Valid	Meaning
`trajectory_from`	`design`	–	design, baseline	Nose used to compute the heating trajectories
`materials`	see text	–	–	Material properties: name -> {label, short (optional), conductivity [W/(m K)], density [kg/m^3], specific_heat [J/(kg K)]}
`shell_material`	`CFRP`	–	–	Shell material (key of materials)
`shell_thickness`	0.0012	m	> 0	Shell thickness
`shell_stations`	`[0.05 m, 0.5 L, 1.0 L]`	–	–	Shell stations: 'x m' from the tip or 'f L' as a fraction of L
`tip_material`	`Al6082`	–	–	Tip insert material (lumped model)
`tip_length`	0.08	m	> 0	Axial length of the tip insert from the apex
`printed_tip_material`	`PETCF`	–	–	Material of the comparison printed tip (1D wall at the stagnation point)
`printed_tip_thickness`	0.004	m	> 0	Wall thickness of the printed tip
`wall_nodes` *	40	–	[5, 400]	Nodes through the wall thickness
`bond_limit`	120	°C	–	Adhesive temperature limit drawn on the shell plot
`bond_label`	`DP490 bond limit`	–	–	Label of the adhesive limit
`printed_tip_hdt`	`[70, 80]`	°C	–	Heat-deflection temperature range of the printed tip polymer

`bluntness`: Tip bluntness

Parameter	Default	Unit	Valid	Meaning
`enabled`	`true`	–	–	Evaluate the tip-bluntness study
`study`	`blunt`	–	–	Database study with blunted noses
`profile`	`vonkarman`	–	–	Profile of the bluntness study
`fineness`	4	–	> 0	Fineness of the bluntness study
`report_ratios`	`[0.05, 0.1]`	–	–	Tip radii r_n/R whose drag and apogee cost are reported
`report_mach`	1.4	–	[1, 5]	Mach number at which the drag cost of the tip is reported
`radius_range`	`[0.002, 0.02]`	m	–	Tip radius range of the heat-transfer curve

`robustness`: Robustness

Parameter	Default	Unit	Valid	Meaning
`enabled`	`true`	–	–	Run the robustness study
`fineness`	4.5	–	> 0	Fineness at which all profiles are compared
`reference_profile`	`vonkarman`	–	–	Apogees are shown relative to this profile
`band`	30	m	>= 0	Half-width of the 'equivalent' band in the plot

Parameter	Default	Unit	Valid	Meaning
`scenarios`	see text	–	–	Scenarios: name + any of transonic_ onset, transonic_factor, drag_factor_ scale, mass_model, ground_temperature

`trade_matrix`: Decision matrix

Parameter	Default	Unit	Valid	Meaning
`enabled`	`true`	–	–	Build the weighted trade matrix
`fineness`	4.5	–	> 0	Fineness at which the matrix is built
`weights`	see text	–	–	Criterion weights (should add up to 1)
`manufacture`	see text	–	–	Judgement score per profile (1-5)
`heritage`	see text	–	–	Judgement score per profile (1-5)

`validation`: CFD validation plots

Parameter	Default	Unit	Valid	Meaning
`cone_study`	`cones`	–	–	Database study of sharp cones (compared with Taylor-Maccoll)
`cone_main_ fineness`	4	–	> 0	Cone of the main study added to the validation plot
`grid_study`	`grid`	–	–	Database study of the grid-convergence runs
`grid_profile`	`vonkarman`	–	–	Profile of the grid study
`grid_fineness`	4	–	> 0	Fineness of the grid study
`grid_machs`	`[1.4, 1.2]`	–	–	Mach numbers of the grid study

`cfd`: CFD solver and studies

Parameter	Default	Unit	Valid	Meaning
`cp_dir`	`data/cfd/cp`	path	–	Folder of the wall-pressure files (one .npz per case)
`fields_dir`	`data/cfd/fields`	path	–	Folder of the saved flow fields
`log`	`data/euler_ sweep_log.csv`	path	–	Run log appended by the solver
`workers`	4	–	[1, 64]	Parallel processes
`cfl`	0.85	–	[0.05, 1]	CFL number of the local time stepping
`fallback_cfl` *	0.5	–	[0.05, 1]	CFL of the restart from free stream if a run diverges
`max_iter`	6000	–	>= 100	Iteration limit per Mach number
`fine_iter_factor` *	2	–	[1, 10]	Iteration-limit multiplier on the fine grid
`tolerance`	2e-05	–	> 0	Convergence: relative change of C_D,w over ‘check‘ iterations
`check` *	250	–	>= 10	Iterations between convergence checks

Parameter	Default	Unit	Valid	Meaning
`first_order_iters` *	300	–	>= 0	First-order iterations at the start of the first Mach number
`grids`	see text	–	–	Grid levels: name -> [cells along nose, cells along cylinder, cells normal, first-cell height / R]
`cylinder_length` *	5	R	> 0	Cylinder behind the nose in the CFD domain
`upstream_distance` *	1	R	> 0	Distance of the outer boundary ahead of the tip (scaled by 1 + 0.3 L/D)
`outer_angle` *	72	°	[30, 89]	Asymptotic angle of the outer boundary (hyperbola)
`save_fields` *	`[1.4, 1.2]`	–	–	Mach numbers at which flow fields are saved (studies with save_fields: true)
`studies`	[]	–	–	Euler studies: list of {name, profiles, fineness, tip_radius_ratios, grid, machs, save_fields}

`cad`: CAD export

Parameter	Default	Unit	Valid	Meaning
`profile_csv`	`true`	–	–	Write the profile coordinates of the design (x from the tip, r) in mm
`profile_points`	200	–	[10, 100000]	Points of the profile CSV
`profile_clustering` *	1.6	–	[1, 4]	Clustering exponent towards the tip of the CSV points (1 = uniform)
`stl`	`true`	–	–	Write watertight STL solids
`stl_facets`	128	–	[8, 4096]	Azimuthal facets of the STL
`stl_points` *	220	–	[10, 100000]	Profile points of the candidate STLs
`stl_points_design` *	400	–	[10, 100000]	Profile points of the design STL
`step`	`true`	–	–	Write a STEP solid of the design (needs the gmsh package)
`step_points` *	200	–	[10, 5000]	Profile points through which the STEP B-spline is fitted
`candidates`	`true`	–	–	Also export every profile of the sweep at the design fineness (STL)

`output`: Outputs

Parameter	Default	Unit	Valid	Meaning
`formats`	`[pdf, png]`	–	pdf, png, svg	Figure file formats
`png_dpi` *	200	dpi	[50, 1200]	Resolution of PNG figures
`plots`	`[all]`	–	–	Plots to draw: 'all' or a list of names (see the user guide)
`latex`	`false`	–	–	Also write LaTeX tables and macros (design-report style)
`text_renderer`	`auto`	–	auto, latex, mathtext	Plot text: 'latex' (needs a LaTeX installation), 'mathtext' (built in) or 'auto'

Parameter	Default	Unit	Valid	Meaning
`plot_options`	{}	–	–	Per-plot overrides, e.g. {trajectory: {xlim: [0, 40]}}

4.3 Structured parameters

`nose.profiles` and `nose.custom_profiles`. The sweep compares the profiles listed in `nose.profiles`. The built-in names are those of the CFD database: `cone`, `ogive` (tangent), `parabolic` ($K' = 1$), `power050` and `power075` (power series $n = \frac{1}{2}, \frac{3}{4}$), `lvhaack` (Haack $C = \frac{1}{3}$) and `vonkarman` (Haack $C = 0$). Other members of the same families (and of any family added by a plugin) are defined in `custom_profiles`:

```
nose:
  custom_profiles:
    power060: {family: power, n: 0.6, label: "Power $n=0.6$"}
    haack_c015: {family: haack, C: 0.15, label: "Haack $C=0.15$"}
  profiles: [vonkarman, power060, haack_c015]
```

The families and their parameters are listed by `python -m nosecone profiles`: `cone`, `tangent_ogive`, `parabolic` (K), `power` (n), `haack` (C) and `elliptical`. The `label` (LaTeX allowed) is used in plots and tables. A custom profile has no CFD data until it is computed: set `aerodynamics.compute_missing: true` (Section 5.6).

`nose.fineness` and `nose.tip_radius_ratios`. The sweep is the product profiles $\times$ fineness $\times$ tip radii. The Weisshorn database covers $L/D = 3$–5 for sharp noses; a fineness outside the computed range is accepted, but its wave drag is extrapolated with $C_{D,\mathrm{w}}(L/D)^2$ held constant. Tip radii $r_n/R > 0$ need CFD data for the blunted nose at two fineness ratios or more (otherwise the case is skipped, or computed if `compute_missing` is set).

`heating.materials`. One entry per material; add your own and refer to it by its key:

```
heating:
  materials:
    Ti64: {label: "Ti-6Al-4V", conductivity: 6.7, density: 4430, specific_heat: 526}
  tip_material: Ti64
```

`conductivity` in W/(m K), `density` in kg/m^3, `specific_heat` in J/(kg K); `label` is used in the plots and `short` (optional) in the LaTeX tables. Materials given in a file are added to the default ones (CFRP, Al6082, PETCF).

`heating.shell_stations`. Positions along the shell where the wall temperature is computed: `"0.05 m"` (distance from the tip) or `"0.5 L"` (fraction of the profile length).

`robustness.scenarios`. Each scenario is a name plus any of the following changes to the nominal model; the apogee of every profile is recomputed and shown relative to `robustness.reference_profile`:

```
robustness:
  scenarios:
    - {name: nominal}
```

```
    - {name: "drag rise from M = 0.75", transonic_onset: 0.75}   # Mach of the start of
    the rise
    - {name: "transonic peak +20 %", transonic_factor: 1.2}      # wave drag x 1.2 below
    transonic_match
    - {name: "rest of vehicle drag +20 %", drag_factor_scale: 1.2}  # k x 1.2
    - {name: "LVO masses, 30 C day", mass_model: lvo, ground_temperature: 303.15}
```

`trade_matrix`**.** `weights` gives the weight of each criterion (`performance`: apogee, `integration`: payload length inside the nose, `manufacture`, `heritage`, `stability`: static-margin change); they should add up to one. Performance, integration and stability are scored automatically from 1 (worst) to 5 (best) between the worst and the best profile; `manufacture` and `heritage` are your judgement, one score per profile name.

`cfd.grids` **and** `cfd.studies`**.** A grid level is `[cells along the nose, cells along the cylinder, cells normal to the wall, first-cell height / R]`. A study is a list of CFD runs; each entry is solved for every profile $\times$ fineness $\times$ tip radius, from the first Mach number down (each Mach number starts from the previous solution):

```
cfd:
  studies:
    - {name: main, profiles: [power060], fineness: [3, 4, 5], machs: [1.6, 1.4, 1.2, 1.1]}
    - {name: blunt, profiles: [vonkarman], fineness: [4.5], tip_radius_ratios: [0.05,
    0.1],
       grid: medium, machs: [1.6, 1.4, 1.2], save_fields: false}
```

Optional keys of a study: `tip_radius_ratios` (default `[0]`), `grid` (default `medium`), `save_fields` (save the flow fields at `cfd.save_fields` Mach numbers) and `max_iter` (iteration limit of this study, instead of `cfd.max_iter`). Results go to `cfd.cp_dir`; `python -m nosecone rebuild-database` turns them into the database CSV. The study name becomes the `study` column of the database; `aerodynamics.studies` chooses which studies feed the wave-drag model (`main`: the sweep; `f45`: direct runs at $L/D = 4.5$).

`output.plots` **and** `output.plot_options`**.** `plots` is `[all]` or a list of plot names: `profiles`, `schematics`, `validation`, `cdw`, `cp`, `trajectory`, `breakdown`, `trade`, `robustness`, `bluntness`, `thermal`, `final`, `gallery`, `fields`, `grid`, `renders` (Section 6.3). Plots that need data the study does not have (e.g. `validation` without cone runs) are skipped with a message. `plot_options` overrides cosmetic settings per plot; the axis ranges that depend on the vehicle are automatic unless set:

```
output:
  plot_options:
    trajectory: {xlim: [0, 40], ylim: [0, 2.1]}      # time [s] and Mach axes
    thermal: {xlim: [0, 30], ylim: [0, 175]}         # time [s] and temperature [degC] axes
```

Table 3: Options of `output.plot_options` (defaults in `PLOT_DEFAULTS`, `nosecone/plotting/figures.py`). "auto": chosen from the data. Lengths in mm scale with the diameter.

Plot	Options (default)
`profiles`	`fineness` (baseline), `ylim_mm` (100), `tip_xlim_mm` (60), `tip_ylim_mm` (38), label positions
`cdw`	`fineness` (baseline), `mach_range` (0.6, 1.65), `ylim` (0, 0.125), `fineness_ylim` (0.03, 0.14), `fineness_mach` (1.4)
`cp`	`fineness` (baseline), `machs` (1.1, 1.4), `xlim` (0, 1.35), `ylim` (-0.25, 0.45)
`trajectory`	`xlim`, `ylim` (auto), `mach_bins_max` (auto), `bin_width` (0.1)
`breakdown`	`altitude` (3000 m), `mach_range` (0.1 to the highest tabulated Mach), `annotate_mach` (1.4)
`trade`	`payload_ylim` (0 to 1.5 payload lengths)
`bluntness`	`machs` (1.2, 1.4, 1.6), `guide` ([kW/(m^{2}K), m] of the $r_n^{-1/2}$ guide line; auto)
`thermal`	`xlim`, `ylim` (auto)
`fields`	`machs` (1.4), `xlim` (auto), `ylim` ($\pm 3.2\,R$), `clim` (-0.25, 0, 0.45)
`grid`, `schematics`	`level` (coarse); `fineness` (2, drawing only)
`validation`, `robustness`, `gallery`	Mach range, colours and markers; `gallery`: `mach` (1.4)

5 Running Studies

5.1 Commands

Everything goes through one entry point, `python -m nosecone <command>` (Table 4; `python -m nosecone <command> --help` lists the options of each command).

Table 4: Commands. Typical run times on a laptop with the Weisshorn database.

Command	What it does	Time
`run CONFIG`	The whole study: sweep, sensitivities, robustness, heating, CAD, plots, tables. Options: `--plots trade,cdw`, `--no-plots`, `--latex`, `--output DIR`, `--quiet`	1–5 min
`export CONFIG`	Only the CAD files of the selected design (CSV, STL, STEP)	seconds
`validate CONFIG`	Check the file; `--show` prints all values used	instant
`describe-config`	Every parameter with unit, range and default (`--basic`: hide advanced)	instant
`profiles`	Profile families and built-in profiles	instant
`cfd CONFIG`	Run the CFD studies of `cfd.studies` (`--study NAME`, `--workers N`, `--dry-run`)	3–10 min per geometry
`rebuild-database CONFIG`	Recompute the database CSV from the CFD wall-pressure files	1 min

A run prints its progress and writes it to `run.log`. When it finishes, start with `summary.md`.

5.2 Worked example 1: a single design

Question: how does a Von Kármán nose of 810 mm ($L/D = 4.5$) with a 4.5 mm aluminium tip perform on Weisshorn? The file `configs/example_single_design.yaml` inherits the whole vehicle from `weisshorn.yaml` and only restricts the sweep to that one nose:

```
base: weisshorn.yaml
project:
  name: Weisshorn single design
  output_dir: ../output/example_single_design
nose:
  profiles: [vonkarman]           # the "sweep" is a single nose...
  fineness: [4.5]                 # ...at a single fineness
  design_profile: vonkarman
  design_fineness: 4.5
  design_tip_radius_ratio: 0.05   # 4.5 mm spherical tip (r_n / R)
robustness: {enabled: false}      # nothing to rank with one profile
trade_matrix: {enabled: false}
cad: {candidates: false}          # only the design
output:
  plots: [profiles, trajectory, breakdown, bluntness, thermal, final]
```

```
python -m nosecone run configs/example_single_design.yaml
```

The console shows each step:

```
CFD database: 126 points (euler_results.csv)
Drag factor k = 0.5601 (calibrated: baseline reaches 9000 m)
Sweep: 1 profiles x 1 fineness x 1 tip radii
  sensitivities: -142.5 m/kg, -99.8 m per 0.01 of C_D
Trajectories: M_max 1.47 nominal, 1.56 envelope; apogee 9000 m
Heating [degC] nominal/envelope: T0 119/156, metal tip 51/72, printed tip 104/138, shell
    inner 77/104
Tip r_n/R = 0.05: wave drag +0.6 %, apogee -2 m
CAD export:
  STL nose_vonkarman_LD4.5_rn4.5mm_FINAL.stl: 122624 triangles, open edges 0, volume
    11.828 L
  STEP nose_vonkarman_LD4.5_rn4.5mm_FINAL.step: 1 solid, volume 11.842 L
```

and `summary.md` gives the answer (some columns left out here):

shape	fineness	cd_w_12	cd_w_14	cd_w_16	apogee	d_apogee	mass_g	l3U_mm	bay_mm	dSM_cal
vonkarman	4.5	0.0437	0.0425	0.0414	9060	+60	354	254	907	-0.036

Reading it. The calibration factor $k = 0.56$ scales the drag of the rest of the vehicle so that the *baseline* nose (Von Kármán at $L/D = 4$, the length of the LVS) reaches the 9 km of the LVO. With that vehicle, the 810 mm nose flies 60 m higher than the baseline (`d_apogee`), weighs 354 g, houses 254 mm of the 3U payload in its curved part and lowers the static margin by 0.036 calibre. On a hot day the aluminium tip stays at 72 °C while a printed PET-CF tip would reach 138 °C (Figure 2b). The files to give to Structure are in `cad/`.

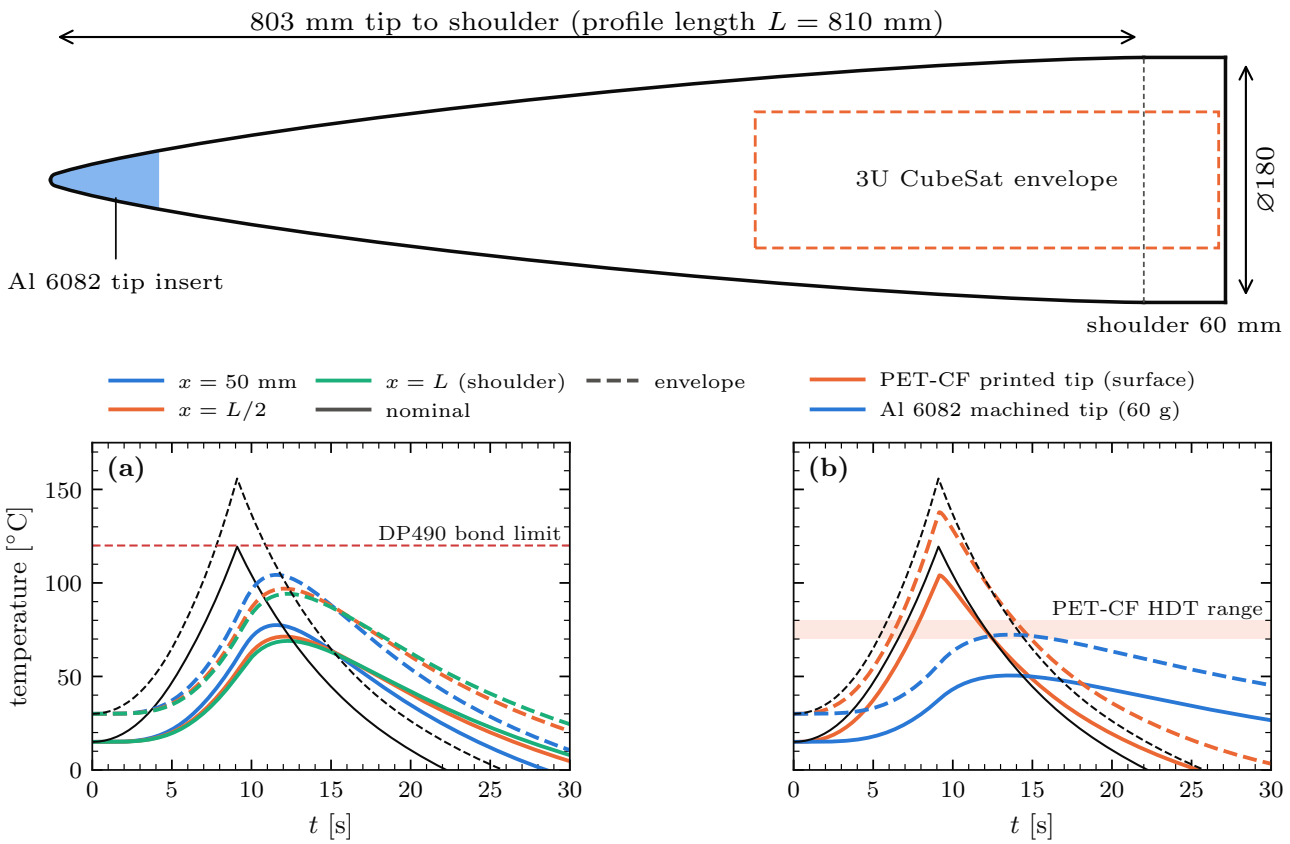


Figure 2: Worked example 1. Top: the design drawn to scale (`fig_final`). Bottom: (a) shell inner-face temperature at three stations, (b) tip temperature (`fig_thermal`); solid: nominal flight, dashed: hot day.

5.3 Worked example 2: a parameter sweep

Question: among the power-$\frac{1}{2}$ (Firehorn), LV-Haack and Von Kármán profiles, which is best on Weisshorn, and how long should the nose be? Any parameter given as a list in `nose.profiles`, `nose.fineness` or `nose.tip_radius_ratios` is swept: here 3 profiles $\times$ 4 fineness ratios.

```
base: weisshorn.yaml
project:
  name: Weisshorn sweep
  output_dir: ../output/example_sweep
nose:
  profiles: [power050, lvhaack, vonkarman]
  fineness: [3.5, 4.0, 4.5, 5.0]
output:
  plots: [cdw, trade, robustness, gallery, schematics]
```

All twelve noses are compared in one table, `summary.md` (sorted by apogee; here the first rows):

```
| shape     | fineness | cd_w_14 | apogee | d_apogee | mass_g | l3U_mm | bay_mm | dSM_cal |
| vonkarman | 5        | 0.0350  | 9099   | +99      | 386    | 282    | 968    | -0.071  |
| power050  | 5        | 0.0387  | 9075   | +75      | 394    | 241    | 1009   | -0.067  |
| vonkarman | 4.5      | 0.0425  | 9060   | +60      | 354    | 254    | 907    | -0.036  |
| power050  | 4.5      | 0.0457  | 9038   | +38      | 361    | 217    | 943    | -0.032  |
| lvhaack   | 5        | 0.0410  | 9036   | +36      | 411    | 355    | 960    | -0.125  |
| ...
```

and on the same plots (Figure 3). **Reading it.** From $L/D = 4$ upwards the Von Kármán nose flies highest (8 to 24 m above the power-$\frac{1}{2}$ nose); at $L/D = 3.5$ the power-$\frac{1}{2}$ nose is slightly

better (6 m). The LV-Haack nose is 63–81 m lower but houses more of the payload (`13U_mm`). Each extra 90 mm of length ($\Delta L/D = 0.5$) is worth less apogee than the previous one (+85, +60, +39 m for the Von Kármán nose). The robustness plot shows that, at $L/D = 4.5$, the ranking does not change under any of the seven modelling scenarios, so the choice does not depend on the uncertain transonic model. Whether to take $L/D = 4.5$ or 5 is then a vehicle-length question (`bay_mm`), not an aerodynamic one.

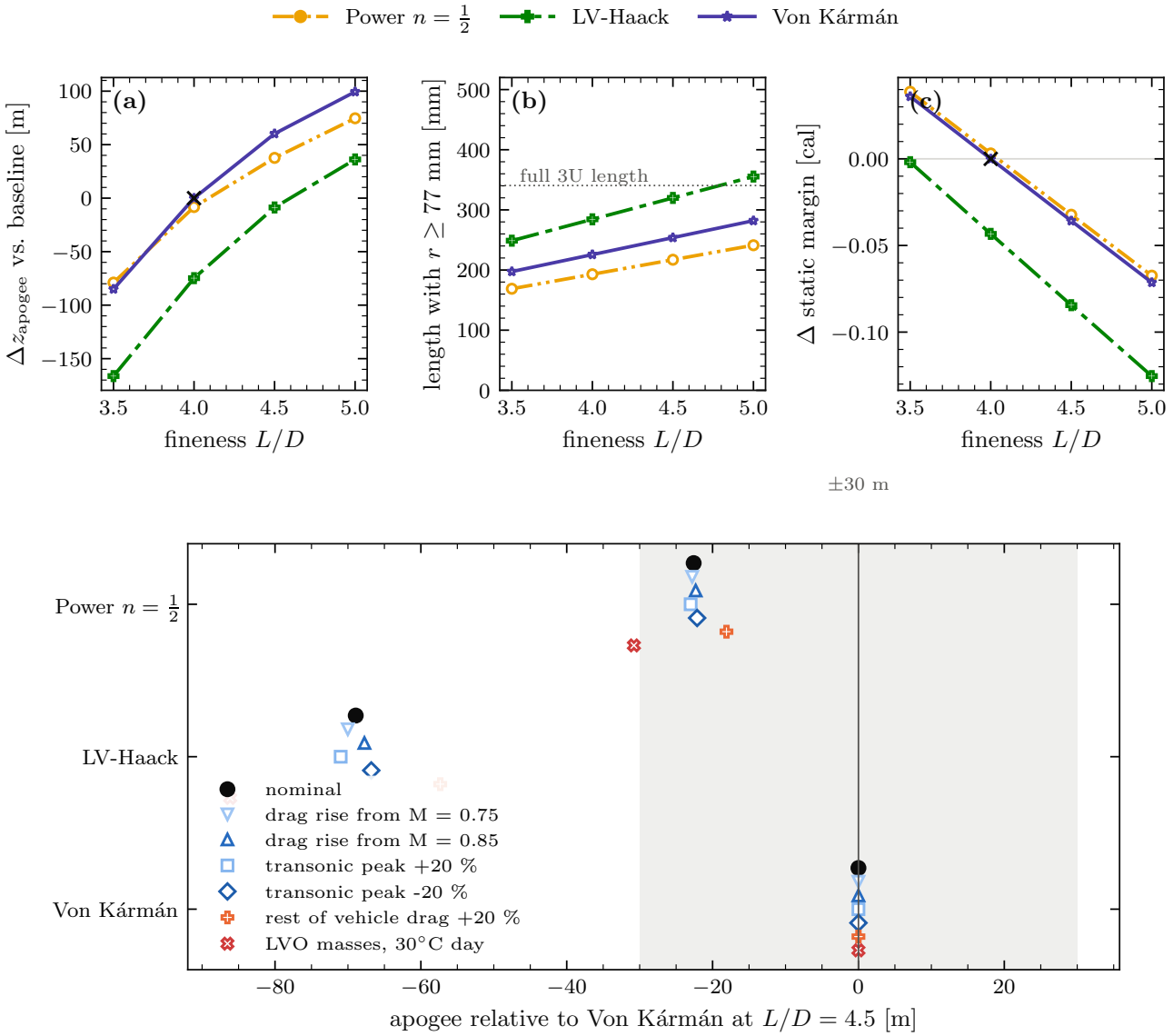


Figure 3: Worked example 2. Top: apogee change, payload length and static-margin change versus fineness (`fig_trade`; the cross is the baseline). Bottom: ranking under the robustness scenarios (`fig_robustness`).

5.4 The full Weisshorn study

`configs/weisshorn.yaml` is the configuration of the design report: seven profiles × five fineness ratios, all plots, and the LaTeX tables and macros of the report (`output.latex: true`, written to `output/weisshorn/latex/`). `python tests/verify_reference.py` checks that these outputs are still identical to the report.

5.5 Another rocket

1. Copy `configs/example_new_rocket.yaml` (a complete example for a 152 mm rocket) to `configs/my_rocket.yaml` and set `project.name` and `project.output_dir`.
2. Fill in `vehicle`, `engine`, `fins` and `launch` from your LVO/LVS.
3. **Calibration.** If you trust a reference apogee for a reference nose (e.g. from RocketPy or a previous flight), keep `flight.calibrate: true` and set `flight.target_apogee`, `nose.baseline_profile` and `nose.baseline_fineness`. Otherwise set `flight.calibrate: false`: the empirical drag build-up is used as is ($k = 1$); apogees are then less accurate, but the comparison between noses remains meaningful.
4. Choose the noses: `nose.profiles`, `nose.fineness`, and the selected design (`design_*`).
5. Set or disable the payload (`payload.enabled`), check the structure and heating inputs, and adapt `trade_matrix.manufacture` and `heritage` to your team.
6. `python -m nosecone validate configs/my_rocket.yaml`, then `run`.

The CFD database is non-dimensional (lengths scaled by the base radius), so it holds for any diameter: only profiles, fineness ratios or tip radii outside it require new CFD.

5.6 New CFD cases

There are two ways to add wave-drag data.

- **On demand**: `aerodynamics.compute_missing: true`. Before the sweep, the tool solves every profile / tip radius that has no data, at the smallest and largest fineness of the sweep and at the design fineness, for `aerodynamics.missing_machs`, in parallel (`cfd.workers`). Set `aerodynamics.user_database` to keep these results in a separate CSV.
- **Planned studies**: list them in `cfd.studies`, run `python -m nosecone cfd CONFIG` (add `--dry-run` first to see the job list), then `python -m nosecone rebuild-database CONFIG`.

One Mach number takes 1–5 minutes on the medium grid (160×80 cells), more near $M = 1.1$; a geometry with four Mach numbers takes 5–10 minutes per CPU core. The solver is deterministic: the same case gives the same result to the last bit.

6 Reading the Outputs

6.1 The output folder

```
output/<study>/
|-- summary.md          the whole sweep in one table, robustness, heating, design
|-- run.log             what the run did (same as the console)
|-- config_used.yaml    every value used, defaults included
|-- tables/             CSV/JSON/TXT results (below)
|-- figures/            one PDF + PNG per plot
|-- cad/                profile CSV, STL and STEP files
|-- latex/              LaTeX tables and macros (if output.latex)
```

6.2 Tables

`trade_table.csv`**: the sweep.** One row per nose (profile × fineness × tip radius), the same content as the table of `summary.md` with full precision:

Column	Meaning
`shape`, `fineness`	profile name and L/D (`tip_radius_ratio` is added if tip radii are swept)
`L_mm`, `V_L`, `S_m2`	nose length [mm], internal volume [L], wetted area [m^2]
`mass_g`	shell (wetted area × areal density) + tip insert [g]
`base_angle`	angle between the profile and the body at the shoulder [°]; 0 = tangent, otherwise a kink
`xcp_over_L`	Barrowman centre of pressure of the nose, from the apex, as a fraction of L
`l3U_mm`, `bay_mm`	length of the curved nose wide enough for the payload; nose-bay length (curved part + the cylinder the payload still needs, at least the shoulder) [mm]
`cd_w_11` ...	nose wave-drag coefficient at $M = 1.1$, 1.2, 1.4, 1.6 (reference area $\pi D^2/4$)
`apogee`, `mach_max`	apogee [m] and maximum Mach number of the nominal flight with this nose
`dSM_cal`	change of static margin relative to the baseline nose [calibres]; negative = less stable
`d_apogee`, `d_bay_mm`	apogee and nose-bay length relative to the baseline nose [m], [mm]

Other tables.

- `robustness.csv`: apogee of every profile in every scenario and `dz_vs_vk`, its difference from the reference profile in the same scenario [m].
- `catalogue.csv`: every profile at the design fineness (volume, area, angles, CP, wave drag computed directly at that fineness when available, apogee changes, mass, payload room, static margin).
- `trade_matrix.csv`: scores (1–5) of each criterion and the weighted total.
- `bluntness.csv`: wave-drag increase [%] and apogee change [m] of each reported tip radius.
- `heating.json`: peak temperatures [°C] (nominal, envelope) of the stagnation point, metal tip, printed tip and shell inner face, and the maximum Mach numbers.
- `design.csv`: geometry of the selected design: profile length L, physical length after blunting, truncation, tangency point of the tip sphere, volume, area, CP, shoulder angle.
- `calibration.txt`: the factor k; `sensitivity.txt`: apogee change per kg and per unit C_D.

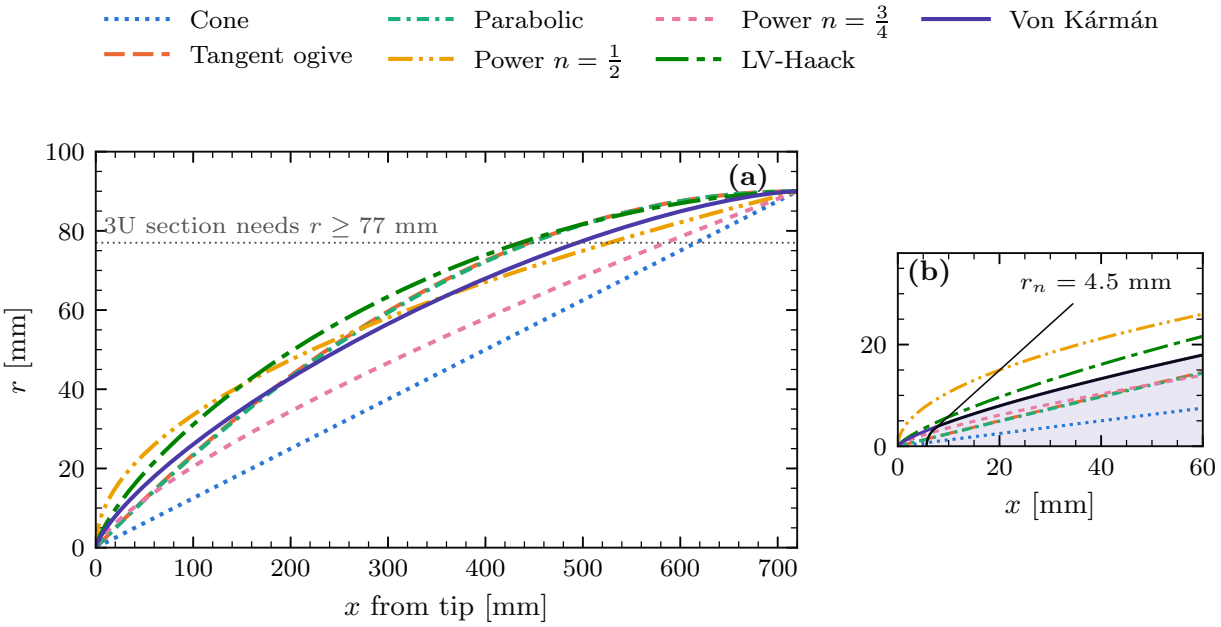
CAD files (`cad/`). Units are millimetres, x from the (blunted) tip. `nose_profile_final.csv`: the profile (x, r), points clustered at the tip, for CAD sketches or templates. `nose_<profile>_LD<L/D>.stl`: every profile of the sweep at the design fineness; `..._FINAL.stl` and `..._FINAL.step`: the selected design. All solids include the shoulder and a flat base; the STL files are checked to be closed (no open edges) and the STEP solid is checked by reading it back. STEP is the file to import in SpaceClaim/Fluent or CAD.

LaTeX (`latex/`). `tab_trade.tex`, `tab_matrix.tex`, `tab_catalogue.tex`, `tab_check45.tex` and `macros.tex` (every number quoted in the design report, e.g. `\FINAPO`). `\input{}` them in a report so that the text always matches the results.

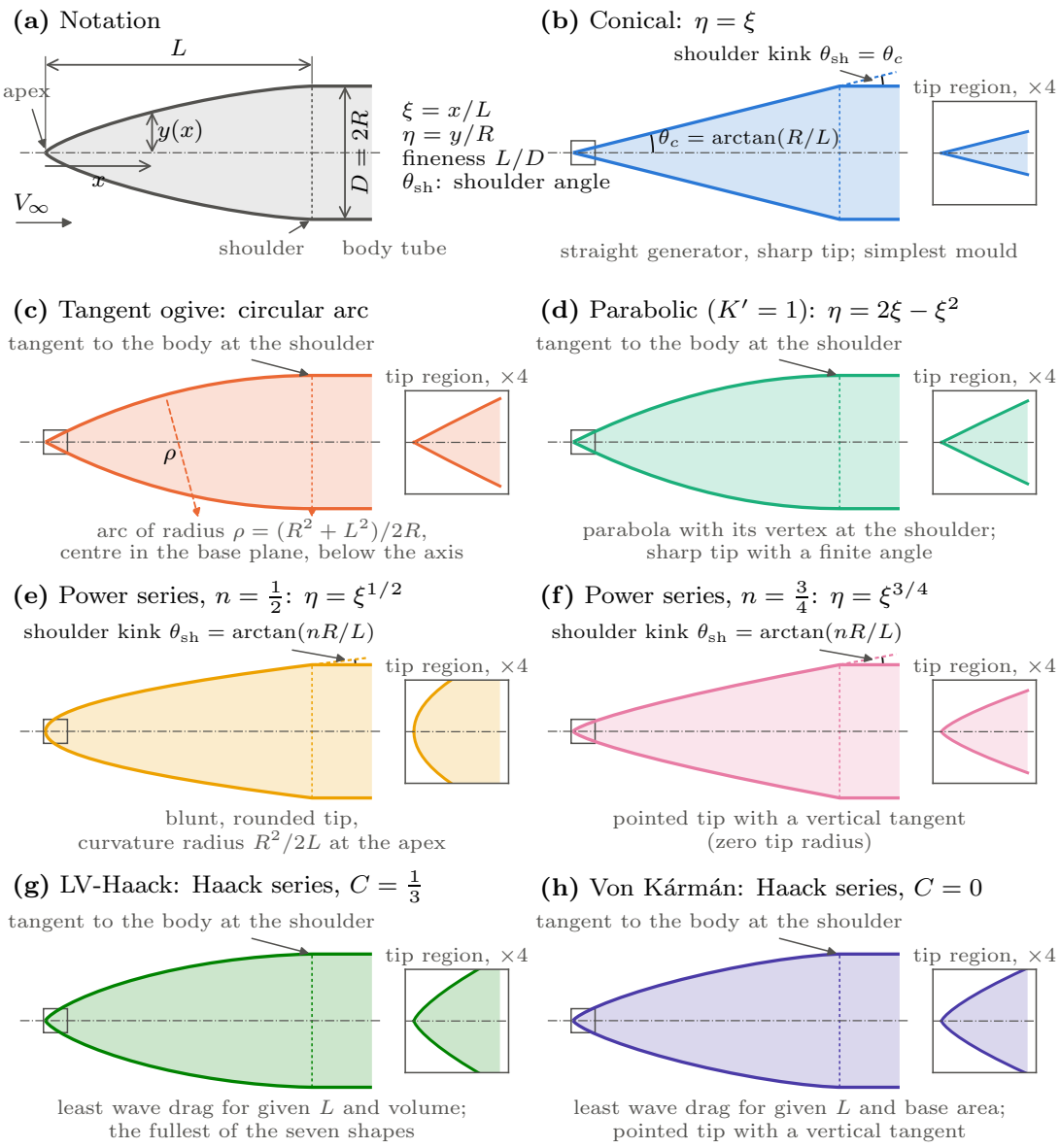
6.3 Plots

The figures below are those of the Weisshorn study. Every plot uses the same colour, line style and marker per profile, so that profiles can be followed from one figure to the next, also in black and white.

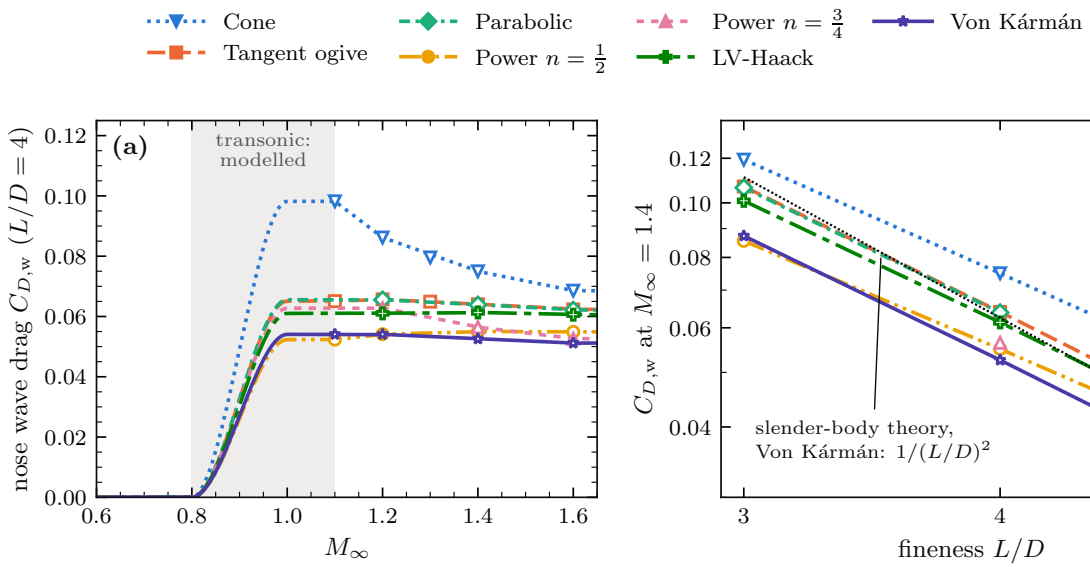
`profiles`. (a) Profiles at the baseline fineness, drawn to scale; the dotted line is the radius the payload needs, so the part of each curve above it can house the payload. (b) Zoom on the tip with the selected spherical blunting shaded. Use it to see how quickly each profile gains radius.



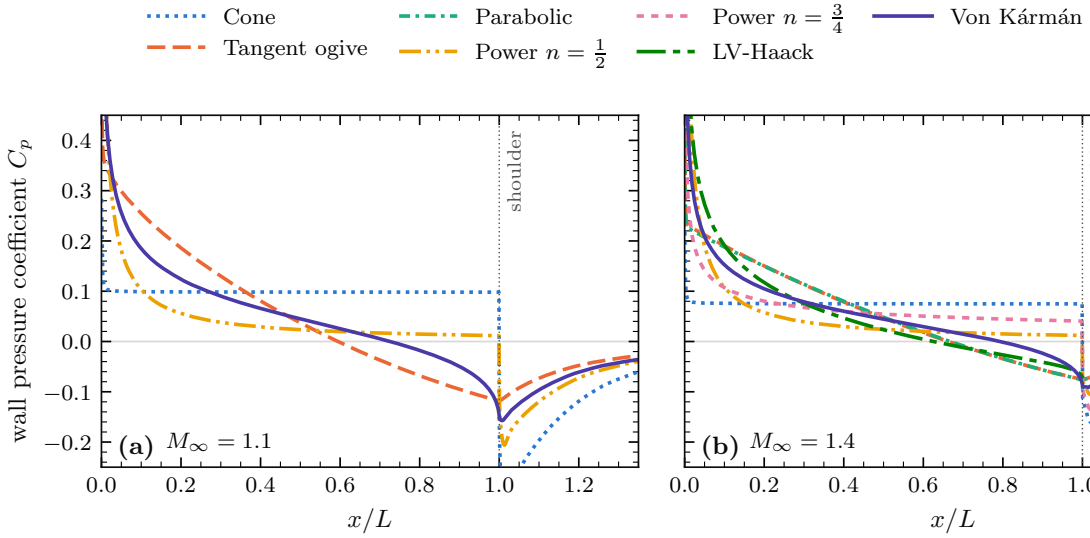
`schematics`. How each profile is constructed (drawn at $L/D = 2$ so that the differences show): upper label, the shoulder (tangent or kink); lower label, the generator and the tip; inset, the tip magnified. Panel (a) defines the notation.



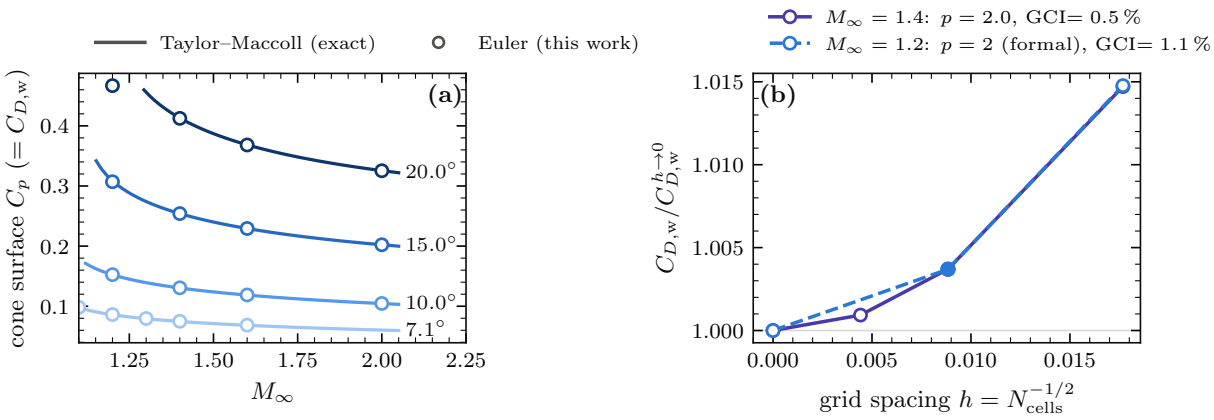
`cdw`. (a) Nose wave drag versus Mach number at the baseline fineness: markers are CFD results, lines the model used in the trajectory; in the grey band the model is an assumption. (b) Wave drag versus fineness at $M = 1.4$ on logarithmic axes; parallel straight lines mean $C_{D,\mathrm{w}} \propto (L/D)^{-2}$, as slender-body theory predicts (dotted line). Lower is better.



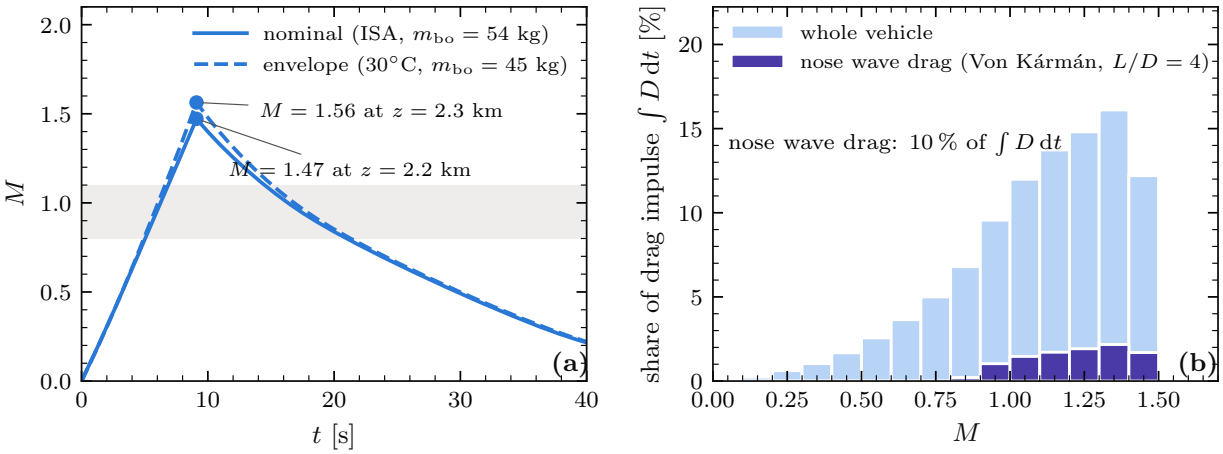
`cp`. Wall pressure coefficient along the nose at two Mach numbers ($x/L = 1$: shoulder). Positive C_p pushes the nose back (drag), negative C_p on the aft part pulls it forward. Explains *why* a profile has low drag.



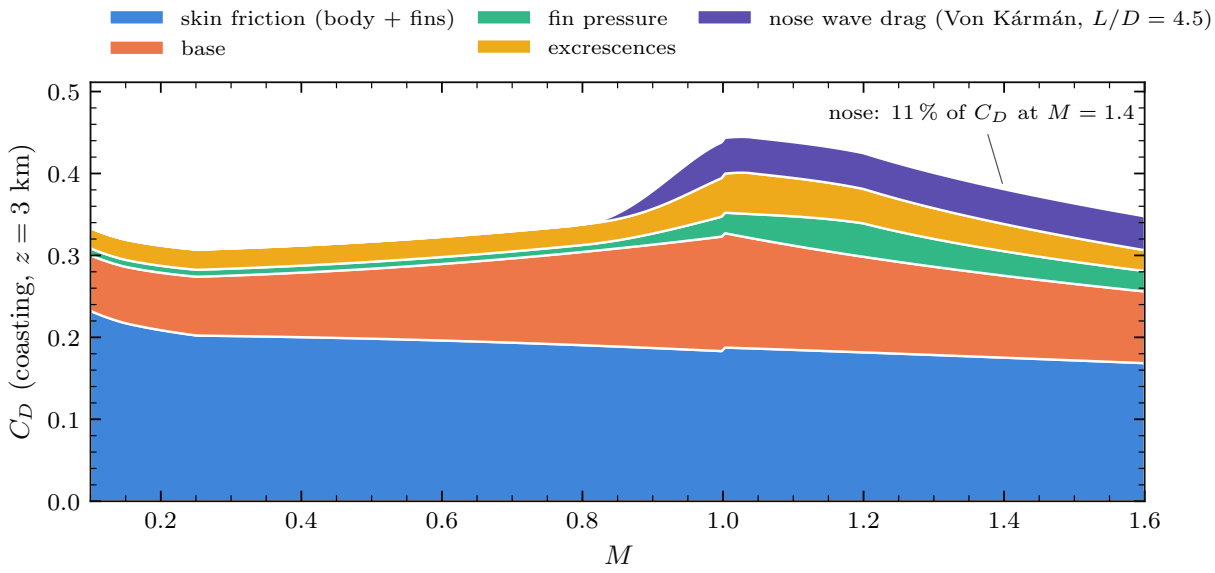
`validation`. Checks of the CFD solver. (a) Cone surface pressure from the solver (circles) against the exact Taylor–Maccoll solution (lines): the circles must sit on the lines. (b) Wave drag on three grids divided by the extrapolated value: the legend gives the observed order p (should be close to 2) and the grid convergence index (GCI, the estimated grid error of the production grid).



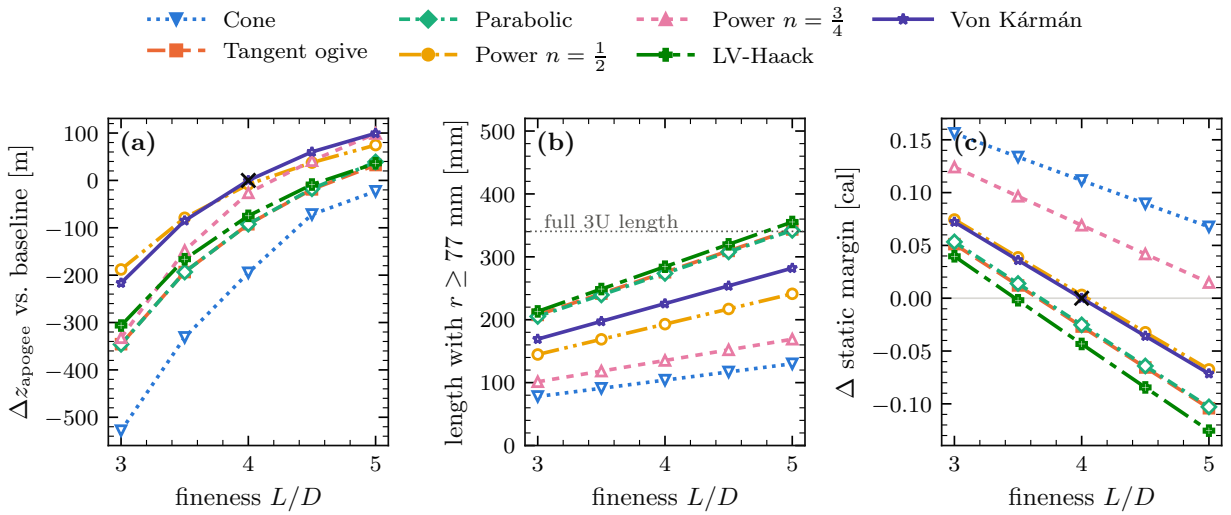
`trajectory`. (a) Mach number versus time on the nominal and hot-day flights, with the maximum Mach number; grey: transonic range. (b) Share of the momentum lost to drag in each Mach interval (light) and the part due to the nose wave drag (dark). The nose shape only matters in the Mach range where the dark bars are.



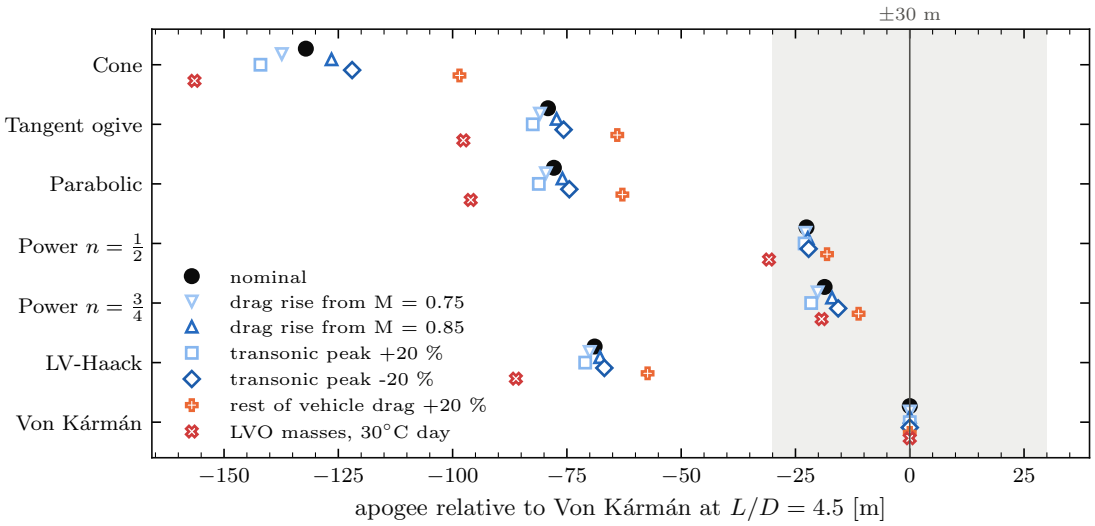
`breakdown`. Drag coefficient of the whole vehicle split into its components (coasting, at the altitude given in the plot options). The nose wave drag is the top band; its share at $M = 1.4$ is annotated. This is why nose shape changes the apogee by tens of metres, not kilometres.



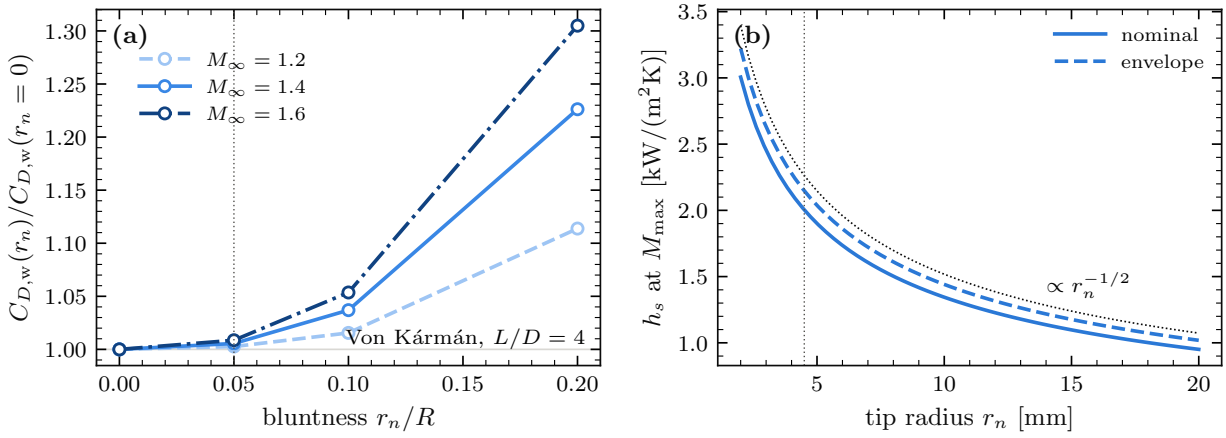
`trade`. The sweep on one figure: (a) apogee change relative to the baseline (cross), (b) length of the nose able to house the payload (dotted: full payload length), (c) static-margin change, all versus fineness, one line per profile. Look for the profile that is highest in (a) for an acceptable (b).



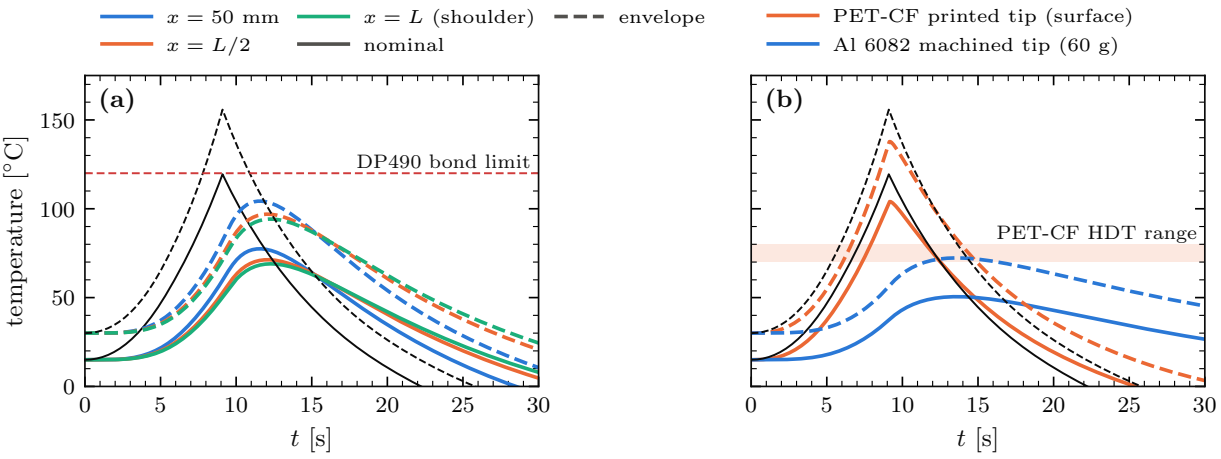
`robustness`. Apogee of each profile relative to the reference profile under each modelling scenario (one marker per scenario). If a profile stays on the same side of zero in every scenario, the ranking does not depend on the uncertain assumptions. Grey band: differences too small to matter (`robustness.band`).



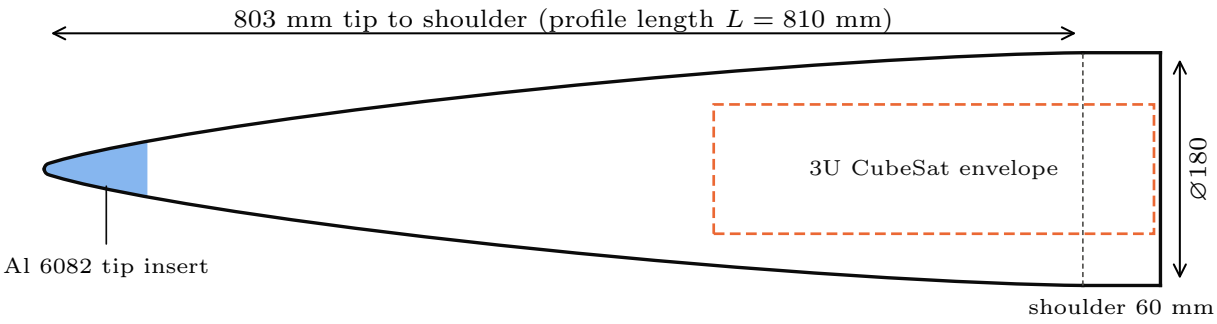
`bluntness`. (a) Wave drag of the blunted nose relative to the sharp one versus tip radius, at three Mach numbers (from the CFD bluntness study). (b) Stagnation-point heat-transfer coefficient at maximum Mach versus tip radius ($\propto r_n^{-1/2}$). A larger tip lowers the heating but raises the drag; the dotted line is the selected radius.



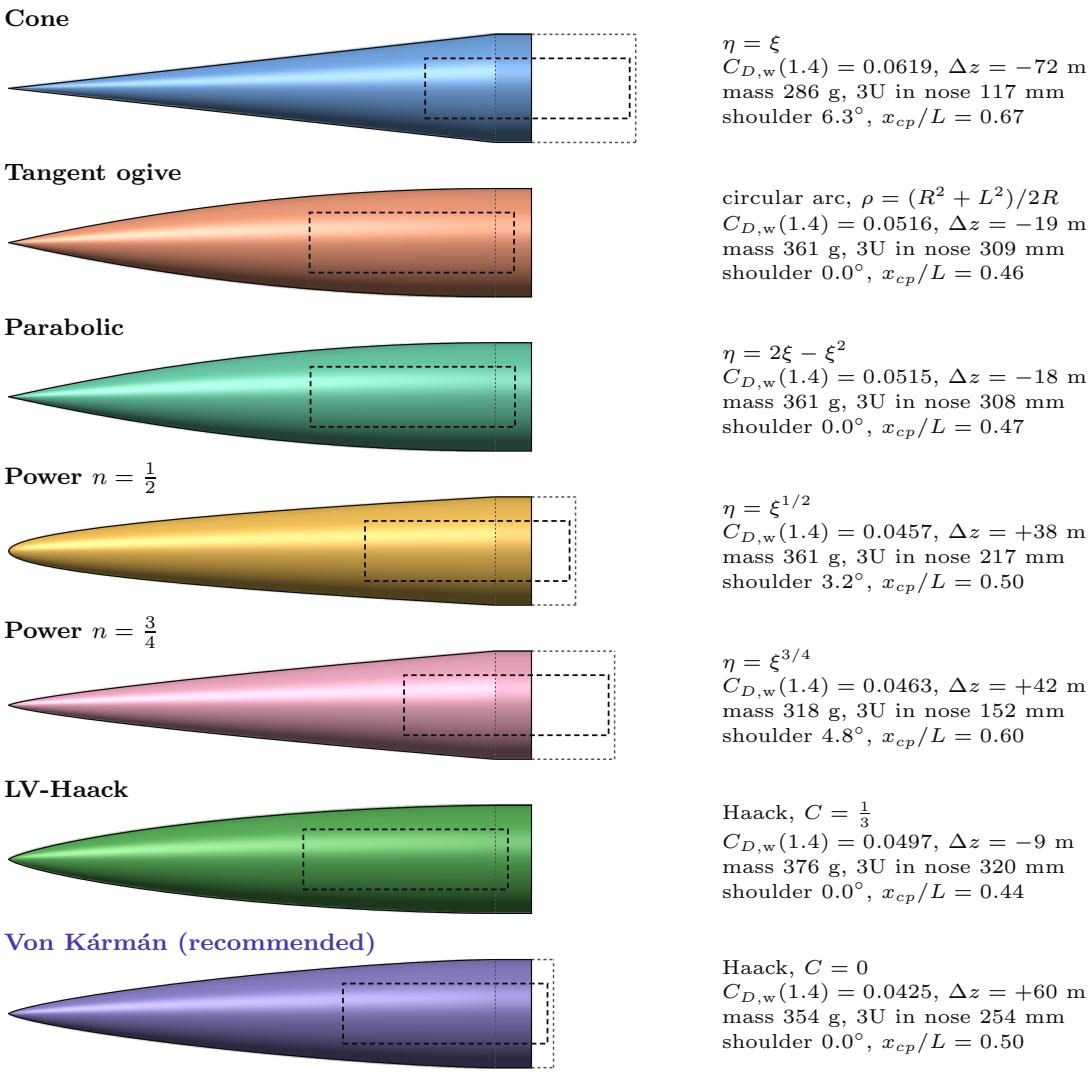
`thermal`. (a) Inner-face temperature of the shell at the configured stations (solid: nominal flight, dashed: hot day; black: stagnation temperature, the upper bound) with the adhesive limit. (b) Tip temperature for a printed polymer tip and for the metal insert, with the polymer's heat-deflection range. Curves must stay below the limits.



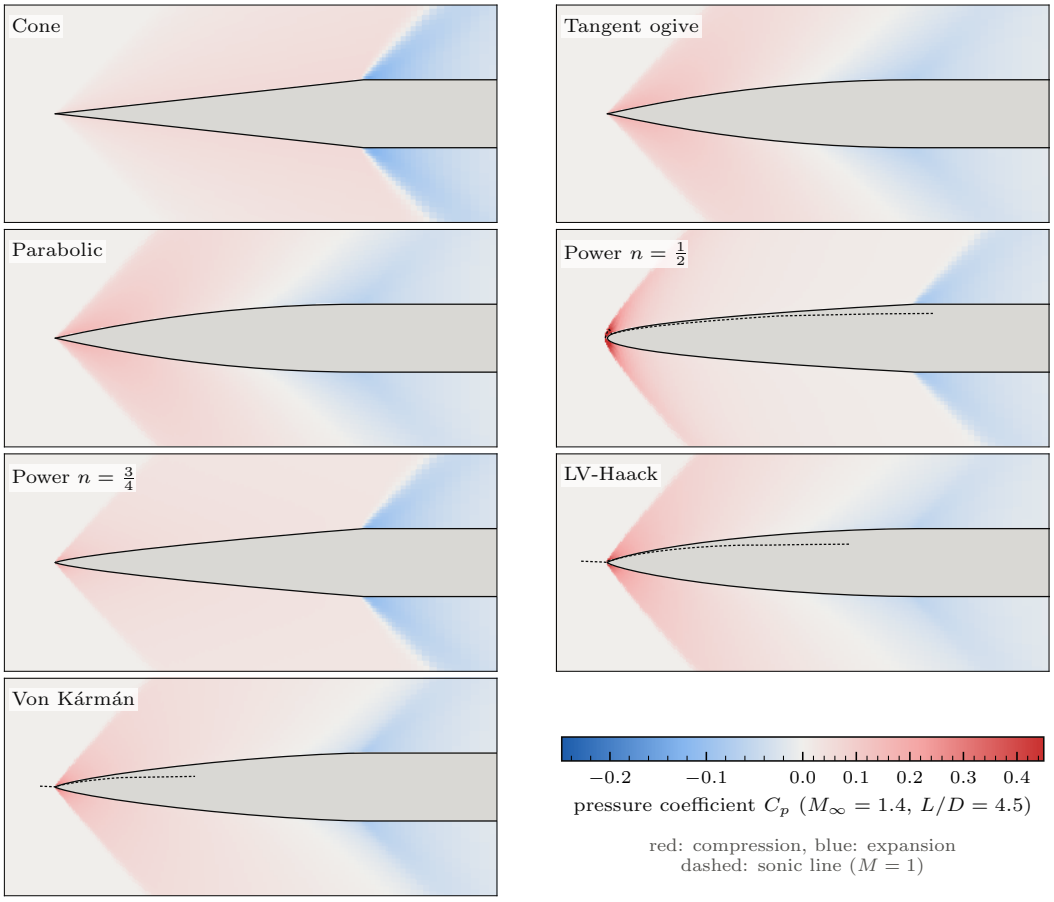
`final`. The selected design to scale: profile, tip insert, shoulder and payload envelope, with the main dimensions.



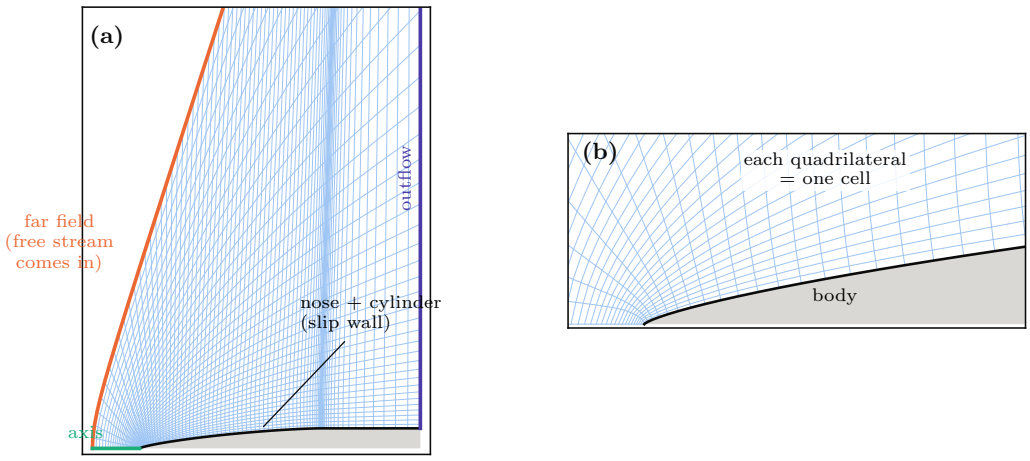
`gallery`. Every profile at the design fineness, to scale, with the payload placed as far forward as the profile allows (dashed box; dotted: extra cylinder needed) and its key numbers.



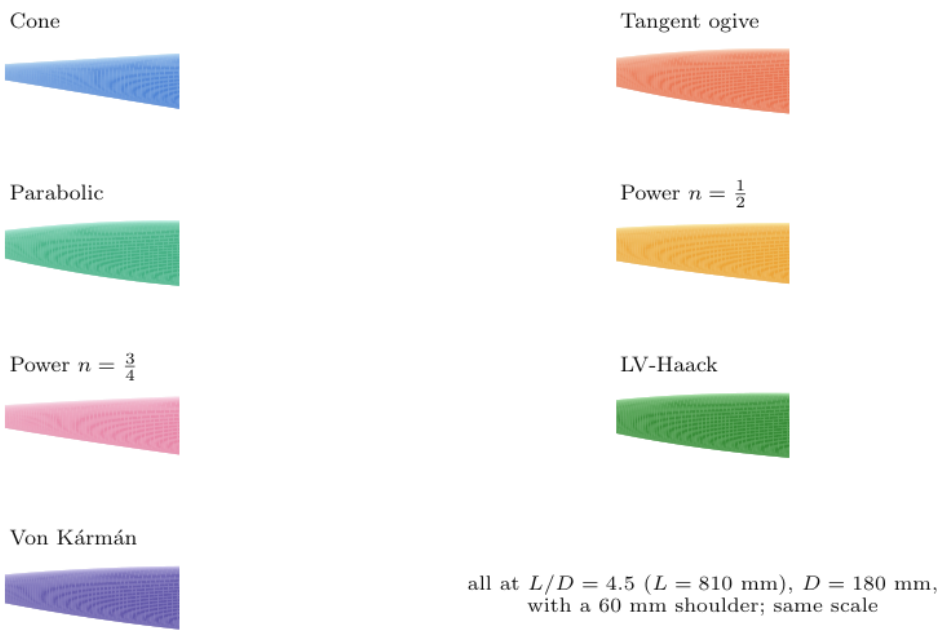
`fields_M14`. Pressure coefficient around each profile from the saved CFD flow fields (red: compression behind the shock, blue: expansion). Dashed: sonic line where a subsonic region exists.



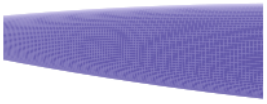
`grid`. The CFD grid (coarse level for clarity) and its boundaries; for explanations, not for decisions.



`renders`. Shaded 3D views of every profile and of the selected design (tip insert in grey), for presentations.



Recommended: Von Kármán, $L/D = 4.5$, 4.5 mm Al 6082 tip insert (grey)



7 Extending the Tool

The code follows a few rules that make extensions safe: every input is a configuration parameter, every fixed number is in `constants.py` with a comment, quantities are in SI units, and every function has a docstring and type hints. After any change, run `python tests/verify_reference.py` (Section 7.5).

7.1 Adding a profile family

A new member of an existing family (another power exponent, another Haack constant) needs no code: declare it in `nose.custom_profiles` (Section 4.3). A new *family* needs one Python function. It can live in a separate plugin file, so that the package itself is not modified. Example: the secant ogive, a circular arc larger than the tangent ogive that meets the body at an angle (file `plugins/secant_ogive.py`):

```
import numpy as np
from nosecone.geometry import register_family

@register_family("secant_ogive", params={"rho_factor": 1.5}, dimensional=True,
                 doc="Secant ogive: arc of radius rho_factor x the tangent-ogive radius")
def secant_ogive(x, L, R, rho_factor):
    """Local radius y(x) [m] of a secant ogive of length L and base radius R."""
```

```
    rho = rho_factor * (R**2 + L**2) / (2 * R)
    alpha = np.arccos(np.sqrt(L**2 + R**2) / (2 * rho)) - np.arctan(R / L)
    x = np.clip(x, 0.0, L)
    return np.sqrt(np.maximum(rho**2 - (rho*np.cos(alpha) - x)**2, 0.0)) - rho*np.sin(
    alpha)
```

- `register_family(name, params, doc, dimensional)` registers the function. With `dimensional=True` it receives x, L, R in metres and returns the radius y; with the default `dimensional=False` it receives $\xi = x/L$ and returns $\eta = y/R$ (simpler for most families): `def my_family(xi, a): return xi**a * (2 - xi**a)`.
- `params` lists the shape parameters with their default values; they become the keys of the custom profile in the configuration.
- The function must accept NumPy arrays, return $y(0) = 0$ and $y(L) = R$, and be monotonic. Check it with `python -m nosecone profiles` and the `profiles` and `schematics` plots.

Then use it from a configuration (`configs/example_plugin.yaml`):

```
project:
  plugins: [../plugins/secant_ogive.py]
nose:
  custom_profiles:
    secant15: {family: secant_ogive, rho_factor: 1.5, label: "Secant ogive"}
  profiles: [ogive, secant15, vonkarman]
aerodynamics:
  compute_missing: true                          # the new family has no CFD data yet
  user_database: ../output/example_plugin/euler_user.csv
```

Everything else follows automatically: geometry, CAD export, mass, payload fit, stability, heating and plots. The only data a new family needs is its wave drag, which `compute_missing` computes with the Euler solver (two geometries, about 30 minutes for this example on a laptop). To make the family part of the package instead, put the function in `nosecone/geometry.py` next to the others; to give it a fixed name, colour and table label, add it to `BUILTIN` (`geometry.py`) and to `COLORS`, `STYLES`, `MARKERS` (`plotting/style.py`).

7.2 Adding a plot

A plot is a function that receives a `PlotContext` (configuration `ctx.cfg`, study `ctx.study`, sweep table `ctx.trade`, robustness table, trajectories `ctx.traj`) and saves a figure:

```
# nosecone/plotting/figures.py
def fig_mass(ctx: PlotContext) -> None:
    """Nose mass versus fineness for every profile."""
    fig, ax = plt.subplots(figsize=(fs.WIDTH, 0.35 * fs.WIDTH))
    for n in ctx.order():
        c, ls, mk = ctx.sty(n)                          # colour, line style, marker of the
     profile
        d = ctx.trade[ctx.trade["shape"] == n].sort_values("fineness")
        ax.plot(d.fineness, d.mass_g, color=c, ls=ls, marker=mk, label=ctx.label(n))
    ax.set(xlabel=r"fineness $L/D$", ylabel="mass [g]")
    ax.legend(fontsize=7)
    ctx.save(fig, "fig_mass")                           # all formats of output.formats
```

```
PLOTS["mass"] = fig_mass                                    # makes "mass" usable in output.
    plots
```

Cosmetic numbers (positions, colours, fixed ranges) go in `PLOT_DEFAULTS["mass"]` and are read with `ctx.opt("mass", key)`, so that users can override them with `output.plot_options`.

7.3 Adding a configuration parameter

Parameters are fields of the dataclasses in `nosecone/config.py`. Adding one is a single line; it is then validated, documented by `describe-config` (and in the tables of this guide when they are regenerated) and available everywhere as `cfg.<section>.<name>`:

```
@dataclass
class StructureConfig:
    ...
    coupler_mass: float = param(0.120, "kg", "Bonded coupler mass, added to the nose mass"
    , NONNEG)
```

`param(default, unit, description, valid range, choices, advanced)` stores the documentation; `POS`, `NONNEG`, `FRACTION` or `Range(lo, hi)` define the valid range. Use the value in the code (here, in `stability.nose_mass`), never a literal number. Regenerate the guide tables with `python -m nosecone describe-config --latex > docs/user_guide/sections/parameters_tables.tex`.

7.4 Changing a physical model

- **A coefficient** of a correlation (friction, base drag, heating): `constants.py`, where every coefficient is named and commented.
- **The drag build-up**: `flight.DragModel.components()` returns one coefficient per component (`friction`, `fins`, `base`, `excr`, `nose_wave`); add a component to the dictionary and it is included in the total, scaled by k, and shown in the `breakdown` plot.
- **The transonic wave drag**: `aerodynamics.WaveDrag.__call__`. With RANS data, add the points to the database (a study with Mach numbers below 1.1) and lower `aerodynamics.transonic_match`.
- **The atmosphere**: `flight.Atmosphere`; add a model name to the `atmosphere.model` choices.
- **Heating**: `heating.py` (correlations, wall conduction, lumped tip).
- **A robustness scenario type**: scenarios accept `transonic_onset`, `transonic_factor`, `drag_factor_scale`, `mass_model`, `ground_temperature`; a new key is read in `design.Study.robustness()` and allowed in `config.validate()`.

7.5 Checking a change

`python tests/verify_reference.py` reruns the Weisshorn study and compares every number, table and CAD file with the reference outputs of the design report (`--figures` adds the figures, `--cfd` three CFD cases). A change that is not meant to alter the results must pass unchanged. If a change is *meant* to alter them (a better model), the script shows exactly what moved; update the files in `tests/reference/` only once the new results have been reviewed, and say so in the commit message.

A Verification Record

The package was refactored from the scripts that produced the Weisshorn design report (`legacy/scripts/`). Before the refactoring, those scripts were rerun and reproduced every result of the report exactly (tables, quoted numbers, STL files, figures pixel for pixel; three CFD cases bit for bit). The new package was then checked against them:

Output	Result
Sweep table, calibration factor k, sensitivities	byte-identical
LaTeX tables (trade, matrix, catalogue) and macros	byte-identical
CFD database rebuilt from the wall-pressure files	byte-identical
CFD solver: the whole database recomputed (38 geometries, 126 Mach points, 16 flow fields)	bit-identical, with the iteration limits of the original runs (see below)
Profile CSV, 7 of the 8 STL files	byte-identical
Robustness table	identical with the 4-decimal k the old script used (see below)
Figures (16)	15 pixel-identical, one with a single pixel differing by 1/255 (sub-pixel label position)

Differences, all intentional:

- *CFD iteration limit.* The first pass of the original sweep (cone and ogive) and the cone validation study ran with a 9000-iteration limit per Mach number; the sweep script was then changed to 6000 for the later passes. With 6000, eight points (cone and ogive at $M = 1.1$, two validation cones at $M = 1.2$) differ by up to 0.006 in C_p; the current legacy script would not reproduce them either. `configs/weisshorn.yaml` records `max_iter: 9000` for those studies, and with it all 126 points are reproduced bit for bit.
- *Calibration factor.* The old robustness, figure and table scripts re-read k from `calibration.txt`, rounded to four decimals (0.5601), while the sweep used the full value. The package uses the full value everywhere. The robustness apogees change by at most 0.4 m (0.05 m relative to Von Kármán); no rounded number of the report changes.
- *Check table.* In the report's "check at the design fineness" table, the parabolic row said "interp. 4–5"; the data used are $L/D = 3$ and 4, and the value at 4.5 is held from 4. The label now says "scaled from 4". The numbers are unchanged.
- *Ogive STL.* Round-off made the apex radius of the tangent ogive $\approx 10^{-16}$ m instead of 0, leaving a pin-hole of 4×10^{-13} mm at the tip (256 open edges). Radii below 1 nm are now set to zero; the file is closed. The other STL files are unchanged.

B Troubleshooting

- **"skipped . . . : no CFD data for profile . . . "**: the database has no wave drag for that profile or tip radius. Use a profile of the database, or set `aerodynamics.compute_missing: true`.

- **"calibration failed: the target apogee ... is not reached"**: the vehicle cannot reach `flight.target_apogee` for k in `flight.calibration_bracket`. Check masses, thrust and impulse, widen the bracket, or set `flight.calibrate: false`.
- **A plot is skipped or "failed"**: the message in `run.log` says why (missing data for `cp`, `fields` or `validation`; a LaTeX problem: set `output.text_renderer: mathtext`). A failed plot never stops the run.
- **The figures look different from the report**: LaTeX is not installed or not found, so the built-in renderer is used. Install MiKTeX/TeX Live, or ignore it: the content is the same.
- **Results changed after updating Python packages**: differences of order 10^{-12} are expected with other library versions; install the pinned versions of `requirements.txt` for exact reproduction.
- **The CFD solver "diverged"**: it restarts automatically from the free stream with a lower CFL number (`cfd.fallback_cfl`). Persistent problems usually mean a profile with a very blunt or kinked tip; try a finer grid level or a smaller `cfd.cfl`.